SW개발/HW제작 설계서

프로젝트 명 : 공공데이터 AI 데이터셋 자동 표준화 및 검증 플랫폼 개발

2026. 09. 08 작성 · 2026. 10. 12 개정

(26_HC277)

수행 단계별 주요 산출물

단계	산출물	일반 (모바일 APP·Web 등)	응용 소프트웨어 (빅데이터·인공지능 등)	응용 하드웨어 (IoT·로봇 등)	본 설계서
환경 분석	시장/기술 환경 분석서	△	△	△	수록
	설문조사 결과서	△	△	△	-
	인터뷰 결과서	△	△	△	-
요구사항 분석	요구사항 정의서	○	○	○	수록
	유즈케이스 정의서	△	△	△	-
아키텍처 설계	서비스 구성도(시스템 구성도)	○	○	○	수록
	서비스 흐름도(데이터 흐름도)	△	○	△	수록
	UI/UX 정의서	△	△	△	-
	하드웨어/센서 구성도	-	-	○	해당 없음
기능 설계	메뉴 구성도	○	○	○	수록
	화면 설계서	○	○	△	수록
	엔티티 관계도	○	○	△	수록
	기능 처리도(기능 흐름도)	○	○	○	수록
	알고리즘 명세서/설명서	△	○	○	수록
	데이터 수집처리 정의서	-	○	-	수록
	하드웨어 설계도	-	-	○	해당 없음
개발 / 구현	프로그램 목록	○	○	○	수록
	테이블 정의서	○	○	△	수록
	핵심 소스코드	○	○	○	수록

- ※ ○ 필수, △ 선택
- 본 프로젝트는 응용 소프트웨어(빅데이터·인공지능) 유형에 해당하며, 필수 산출물 전체와 시장/기술 환경 분석서를 수록함
- 전용 하드웨어를 쓰지 않으므로 하드웨어 관련 산출물은 작성하지 않음

시장/기술 동향 분석

1. 시장 동향

- AI 학습 데이터 시장은 2024년 약 26억 달러에서 2030년 86억 달러로 연평균 21.9% 성장 전망 (Grand View Research, 2025)
- 생성형 AI·LLM 확산으로 출처와 품질이 확인된 학습·분석용 데이터 수요 증가
- 국내 공공데이터는 OECD 공공데이터 평가(OURdata Index)에서 4회 연속 1위 기록, 개방 건수는 꾸준히 증가
- 개방은 원천 API 제공까지이고, 분석·학습에 바로 쓸 수 있는 표 형태로 가공하는 작업은 이용자 몫으로 남아 있음
- → 공공데이터를 수집·정제·검증해 재사용 가능한 테이블과 데이터셋으로 만들어 주는 도구가 필요한 시점

2. 기술 성숙도 단계

단계	정의	개요
Level 0	수동 처리	개별 스크립트로 API를 호출하고 수기로 정제. 표준·재현성 없음
Level 1	부분 자동화	수집·정제를 스크립트화. 형식 변환 일부만 자동화
Level 2	파이프라인화	수집→정제→검증을 ETL 파이프라인으로 연결 (dbt·Airflow 등)
Level 3	**선언적·재현 빌드**	**명세(BuildSpec) 기반 실행, 출처·Manifest 기록, 스냅샷 관리, 다중 포맷 내보내기 ★ 본 프로젝트**
Level 4	지능형 자동화	스키마 자동 매핑, 품질 자동 개선, LLM 기반 데이터 큐레이션 (지향점)

시장/기술 동향 분석

3. 경쟁 제품 비교

기능	본 프로젝트	공공데이터 포털	HF Datasets	Kaggle	dbt·ETL	DVC
공공 API 직접 수집	○	○	×	×	△	×
자동 정규화·표준화	○	×	×	×	○	×
선언적 빌드(명세)	○	×	×	×	○	△
다중 포맷 내보내기	○	△	○	△	×	×
데이터셋 카드 자동 생성	○	×	△	△	×	×
재현성(Manifest·버전)	○	×	△	×	△	○
허브 배포(HF·Kaggle)	○	×	○	○	×	×
비개발자용 웹 UI	○	△	△	○	×	×
국내 공공데이터 특화	○	○	×	×	×	×

- ○ 제공, △ 일부 제공, × 미제공

4. 차별화 비교

구분	기존 접근	본 프로젝트
공공데이터 포털	원천 API 제공까지. 분석·학습 적합성은 이용자가 확인	정제·검증·분할·문서화를 거친 테이블과 데이터셋으로 산출
범용 ETL 도구 (Airflow 등)	파이프라인 실행은 되나 도메인 지식과 검증 규칙은 사용자 몫	kpubdata가 제공하는 공공 API 접근 위에 스키마 계약·품질 규칙을 명세로 선언
데이터 허브 (Hugging Face 등)	배포·공유 계층. 생성 과정은 남지 않음	생성 규칙·실행 기록·입력 지문을 Manifest로 남기고 카드와 함께 게시
일반 LLM 데이터 도구	LLM이 스키마와 내용을 추정해 생성	LLM 답변을 Builder 근거와 대조하고 사용자 승인 전에는 실행하지 않음

요구사항 정의서

요구사항 ID	요구사항명	기능ID	기능명	세부사항	예외사항	상태
A01	데이터 수집	A01_B01	공공 API 수집	kpubdata로 공공 API를 호출해 원천 레코드를 페이지 단위로 수집·저장	수집 실패 시 해당 소스를 실패로 기록하고 나머지 소스는 계속 처리	구현
		A01_B02	파일·URL 수집	업로드 파일(CSV·JSON·JSONL·Parquet)과 HTTPS URL(JSON·JSONL·CSV)을 소스로 사용	허용되지 않은 형식·주소는 명세 검증에서 거부	구현
		A01_B03	소스 카탈로그	kpubdata에 등록된 제공 기관·데이터셋 목록을 조회	조회 실패 시 오류 상태와 다시 시도 제공	구현
		A01_B04	정기 수집	GitHub Actions 일정에 따라 분야별 데이터를 주기적으로 수집·게시	실패 시 이슈를 자동 등록	구현
A02	데이터 가공·품질	A02_B01	BuildSpec 작성	소스·스키마 계약·품질·분할·내보내기를 하나의 명세로 선언	필수 항목 누락 시 실행 전에 오류 목록 반환	구현
		A02_B02	BuildSpec 검증	필수 항목, 소스 종류, 스키마 계약, 분할, 라이선스 등 15개 영역 검사	위반 사항을 code·path·message로 모두 모아 반환	구현
		A02_B03	결과 미리보기	저장 없이 소스별 스키마와 샘플 행을 확인	미리보기 실패 시 원인을 화면에 표시	구현
		A02_B04	Bronze 원천 보존	수집한 원천 레코드와 수집 시각·파라미터·건수·체크섬을 보존	저장 실패 시 해당 소스를 실패 처리	구현
		A02_B05	Silver 정규화	DuckDB로 결측 토큰·컬럼 병합·이름 변경·자리 채움·형 변환·파생 컬럼을 순서대로 적용	형 변환으로 값이 사라지면 오류 처리	구현
		A02_B06	품질 검사	필수 컬럼, 자료형, 중복률, 결측률, 최소 행 수, 값 범위, 컬럼 간 비교 검사	fail은 Gold 전에 중단, warn은 기록 후 진행	구현
		A02_B07	Gold 선택·분할	행 필터와 컬럼 선택, 비율·키 기준 분할, 두 소스 조인	분할 비율 합이 맞지 않으면 검증 오류	구현

요구사항 정의서

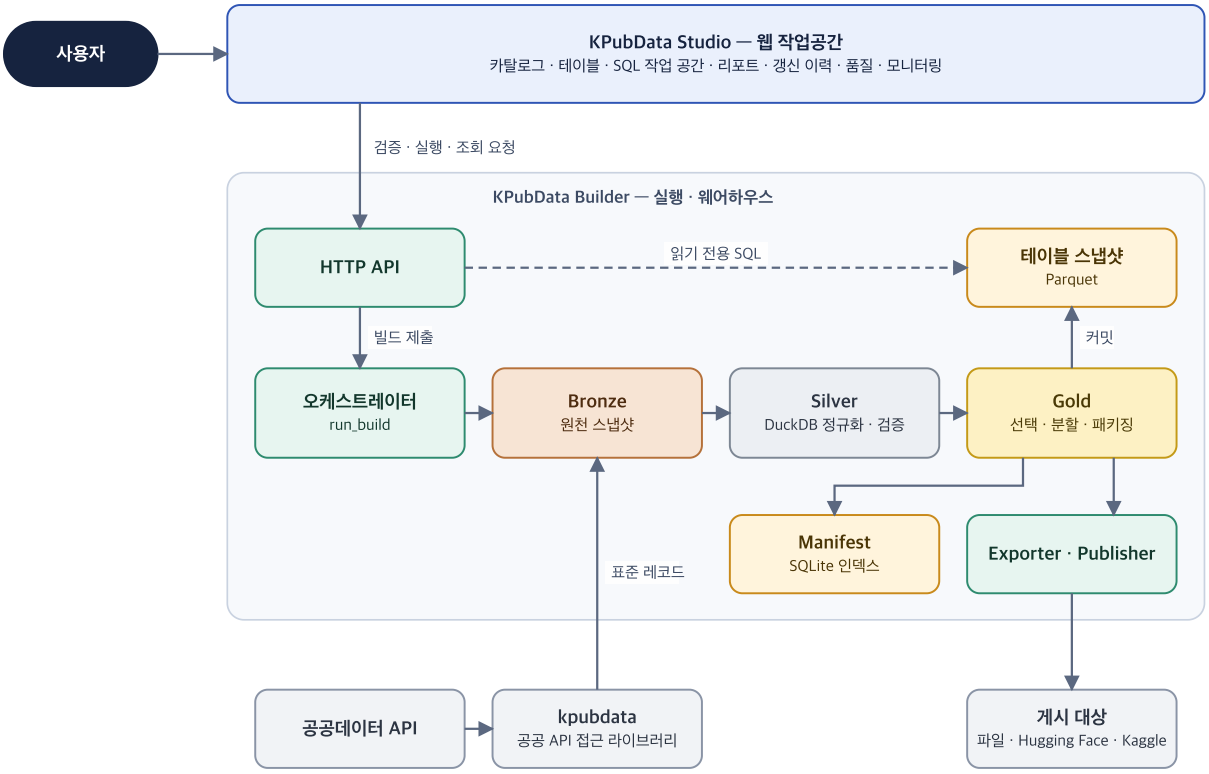
요구사항ID	요구사항명	기능ID	기능명	세부사항	예외사항	상태
A03	산출물 생성·배포	A03_B01	데이터셋 카드	스키마·샘플·출처·처리 내역·개인정보 항목을 담은 README.md와 card.json 생성	빈 항목이 있으면 게시를 막음	구현
		A03_B02	다중 포맷 내보내기	CSV·JSONL·Parquet·Markdown·Hugging Face·Kaggle 형식으로 출력	등록되지 않은 형식은 검증에서 거부	구현
		A03_B03	Manifest 기록	상태·입력·산출물·행 수·출처·품질 결과·실패 내역을 manifest.json으로 기록	Manifest 없이 끝난 실행은 종료 기록으로 따로 남김	구현
		A03_B04	테이블 스냅샷	성공한 실행의 Gold를 웨어하우스 스냅샷으로 커밋하고 최근 3개 유지	동시 갱신 충돌 시 커밋을 거부하고 사유 기록	구현
		A03_B05	외부 게시	산출물을 로컬·Hugging Face·Kaggle로 게시. 준비 상태 점검 후 실행	라이선스·개인정보 검사 미통과 시 게시 차단	구현
A04	웹 작업공간(Studio)	A04_B01	홈	조치가 필요한 테이블, 최근 스냅샷·분석, 연결 문제 표시	조회 실패 시 영역별 오류 표시	구현
		A04_B02	카탈로그·테이블 만들기	소스를 찾아 3단계 마법사로 테이블 생성	검증 실패 시 실행 차단	구현
		A04_B03	테이블·테이블 상세	스냅샷·스키마·프로파일·미리보기·품질 확인	웨어하우스가 없으면 실행 기준 보기로 전환	구현
		A04_B04	SQL 작업 공간	스냅샷에 읽기 전용 SQL 실행, 분석 저장, 집계 차트, 결과 내보내기	허용되지 않은 SQL은 Builder가 거부	구현
		A04_B05	리포트	실행 근거를 바탕으로 리포트 작성, Markdown·HTML 내려받기	브라우저에만 저장됨을 화면에 표시	구현
		A04_B06	갱신 이력·게시	실행 목록·상세·산출물 확인, 재실행, Hugging Face 게시	실행 실패 시 실패 단계와 코드 표시	구현
		A04_B07	연결·설정	제공 기관 자격 증명 관리, Builder 연결 상태·버전 확인	최소 계약 버전 미만이면 경고 배너	구현

요구사항 정의서

요구사항 ID	요구사항명	기능ID	기능명	세부사항	예외사항	상태
A05	운영 자동화	A05_B01	CI	Builder: ruff·mypy·pytest(Python 3.10~3.13). Studio: ESLint·Vitest·Playwright	검사 실패 시 병합 차단	구현
		A05_B02	계약 호환 점검	Builder API 계약과 Studio 스키마의 불일치를 CI에서 검사	불일치 시 실패	구현
A06	데이터 보안·거버넌스	A06_B01	개인정보 검사·마스킹	주민등록번호·전화번호·이메일·사업자번호 패턴과 컬럼명 규칙으로 검사, 선언한 컬럼 마스킹	block 모드에서 검출 시 해당 소스 중단	구현
		A06_B02	라이선스·재배포 판정	소스별 재배포 조건(허용·비상업·금지·미확인) 중 가장 엄격한 값 적용	게시할 때 라이선스 미기재면 검증 오류	구현
		A06_B03	읽기 전용 SQL 검사	단일 SELECT만 허용, 테이블 함수·비결정 함수·외부 접근 차단	위반 시 실행하지 않고 거부	구현
		A06_B04	인증·자격 증명	OIDC(Keycloak) 로그인, 제공 기관 키 암호화 저장	토큰 갱신 실패 시 로그인 화면으로 이동	구현
A07	AI 보조	A07_B01	Ask KPubData	화면 문맥과 Builder 근거를 바탕으로 답변·SQL·다음 동작 제안	근거에 없는 참조는 답변에서 제거	구현
		A07_B02	LLM 스키마 자동 매핑	컬럼명을 표준 용어로 자동 매핑	-	제외
A08	운영 관제·품질 시각화	A08_B01	모니터링	API·대기열·작업자·스냅샷 저장소 상태와 24시간 실행 통계	권한이 없으면 안내 문구만 표시	구현
		A08_B02	품질 센터	WARN·FAIL 결과와 스키마 변화를 테이블 전체에서 조회	점수는 계산하지 않고 검사 결과만 표시	구현
A09	버전 관리	A09_B01	스냅샷·명세 이력	테이블 스냅샷 revision, 명세 개정 이력과 되돌리기	충돌 시 최신 revision 기준으로 거부	구현

- A07_B02는 구현 범위에서 제외하고, 컬럼 대응은 명세의 rename·coalesce로 선언함
- 개인정보 검사는 규칙 기반으로, 버전 관리는 웨어하우스 스냅샷과 명세 개정 이력으로 구현함

서비스 구성도 - 서비스 시나리오



서비스 구성도 - 서비스 시나리오

1. 구성 요소

구성 요소	역할
KPubData Studio	웹 작업공간. 화면과 사용자 흐름 담당. 실행 엔진이 아니며 Builder API만 호출
KPubData Builder	실행·웨어하우스 계층. 명세 검증, Bronze·Silver·Gold 실행, 스냅샷·Manifest 기록, 읽기 전용 SQL 제공
kpubdata	공공 API 접근 라이브러리. 제공 기관별 호출과 표준 레코드 변환 담당
저장소	실행 작업공간(로컬 파일), 웨어하우스 스냅샷(Parquet), SQLite 인덱스
외부 대상	Hugging Face·Kaggle 등 게시 대상, OIDC 인증 제공자(Keycloak)

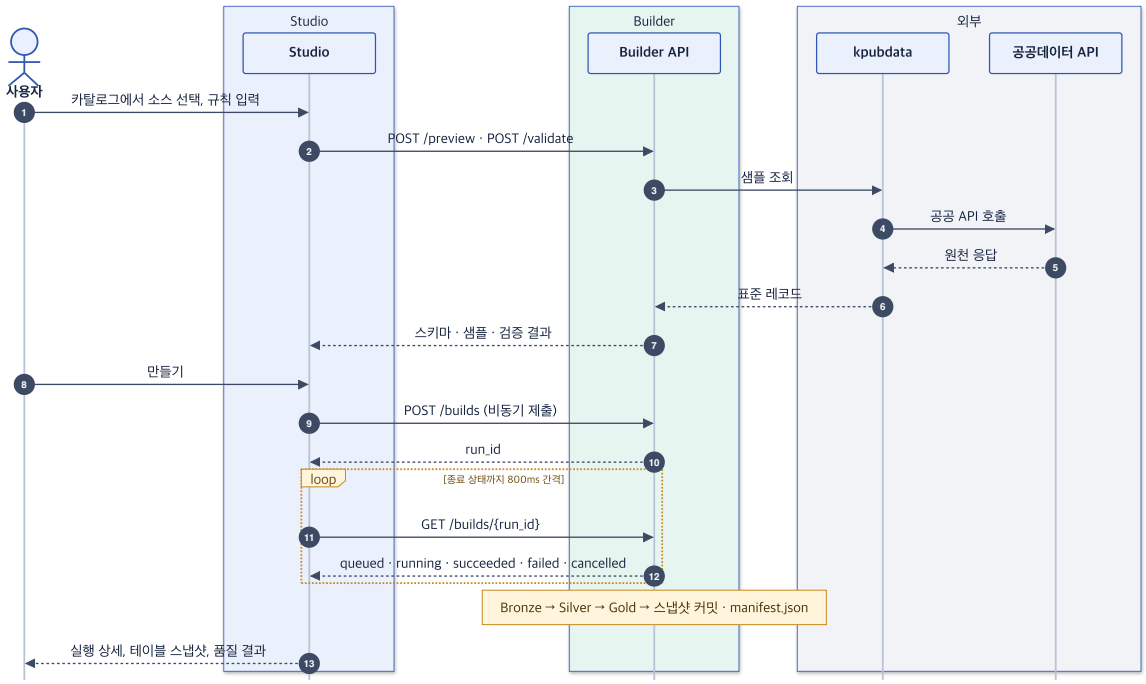
2. 테이블 만들기 (Studio 기반)

- ① 사용자가 Studio 카탈로그에서 소스를 고르고 규칙(BuildSpec)을 구성
- ② Studio가 Builder API에 미리보기와 검증을 요청해 스키마와 샘플 확인
- ③ 만들기를 실행하면 Builder가 Bronze→Silver→Gold로 가공하고 스냅샷과 Manifest 생성
- ④ 사용자는 테이블 상세, SQL 작업 공간, 리포트에서 결과 활용

3. 정기 수집·게시 (일정 기반)

- ① GitHub Actions 일정이 분야별 수집 작업을 주기적으로 시작 (기상 3시간, 관광·의약품 주 1회, 부동산 월초)
- ② kpubdata가 공공 API에서 데이터를 수집하고 게시 스크립트가 정제·패키징
- ③ 검증을 통과한 데이터셋을 Hugging Face에 게시
- ④ 실패하면 이슈를 자동 등록해 담당자가 확인
- 정기 게시는 기존 게시 스크립트 경로로 동작하며, BuildSpec 기반 경로로 순차 이전 중임

서비스 흐름도



- ① 규칙 작성 — 사용자가 카탈로그에서 소스를 선택하고 파라미터와 규칙 입력
- ② 미리보기·검증 — Studio가 `POST /preview`, `POST /validate`를 동시에 요청
- ③ 원천 조회 — Builder가 kpubdata를 통해 공공 API에서 샘플을 받아 표준 레코드로 변환
- ④ 실행 제출 — `POST /builds`로 비동기 제출하고 `run_id` 수신
- ⑤ 진행 확인 — `GET /builds/{run_id}`를 800ms 간격으로 조회 (탭이 숨겨지면 중단)
- ⑥ 결과 확인 — 실행 상세에서 단계별 상태·품질·이벤트 확인, 테이블 스냅샷 활용

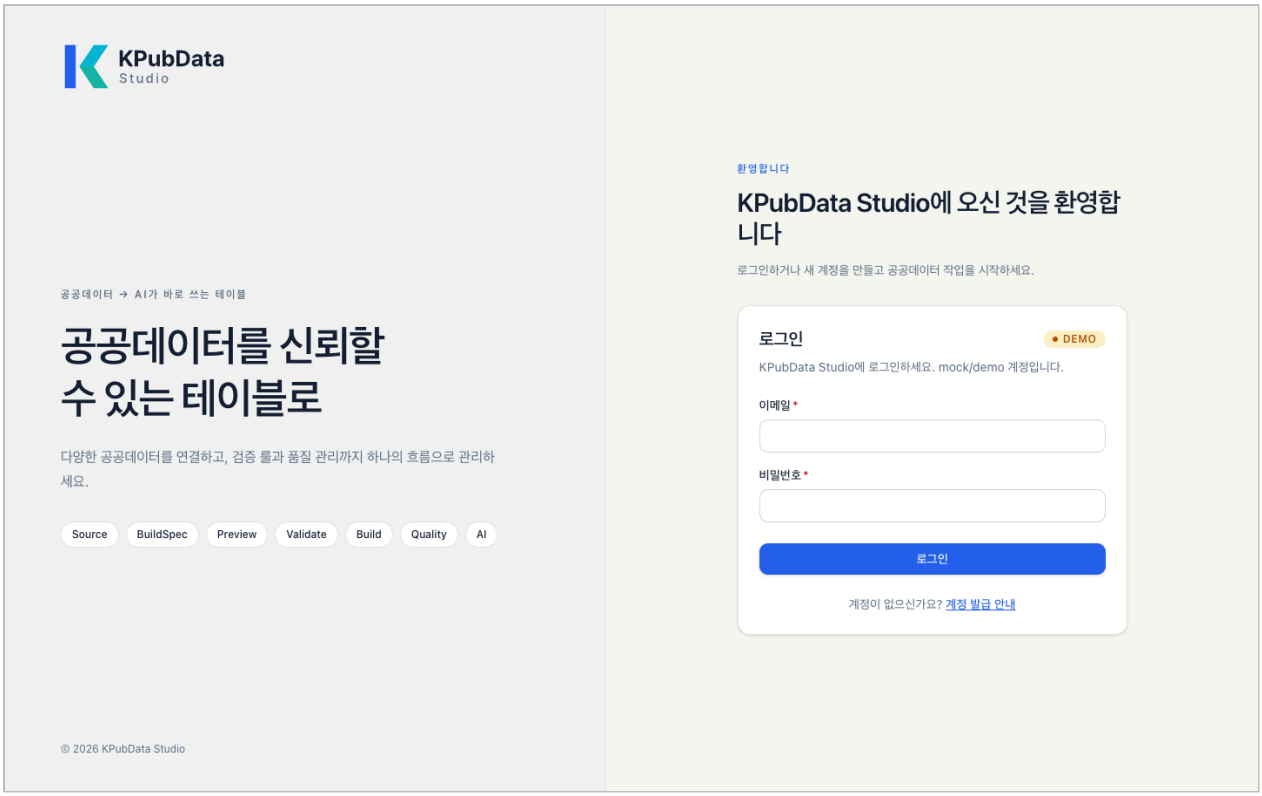
메뉴 구성도



그룹	메뉴	경로	설명
-	홈	`/`	조치가 필요한 테이블과 최근 변경
데이터	카탈로그	`/discover`	공공 API 소스 데이터셋 탐색
	테이블	`/tables`	소스로 만든 테이블 목록과 상세
분석	SQL 작업 공간	`/sql`	테이블 스냅샷에 SQL 실행
	저장된 분석	`/analyses`	스냅샷에 고정해 저장한 분석
	작업대	`/workspace`	최근 작업과 저장한 BuildSpec
	리포트	`/reports`	근거 기반 리포트 작성
운영	갱신 이력	`/refresh-jobs`	테이블을 만들고 갱신한 실행 기록
	품질	`/quality`	품질 검사 결과와 스키마 변화
	모니터링	`/monitoring`	Builder 상태와 실행 통계
	관리	`/admin`	운영 정책·전체 실행·가입 승인 (관리자 전용)
하단	연결	`/connections`	제공 기관 자격 증명
	설정	`/settings`	Builder 연결, 계정, AI 키

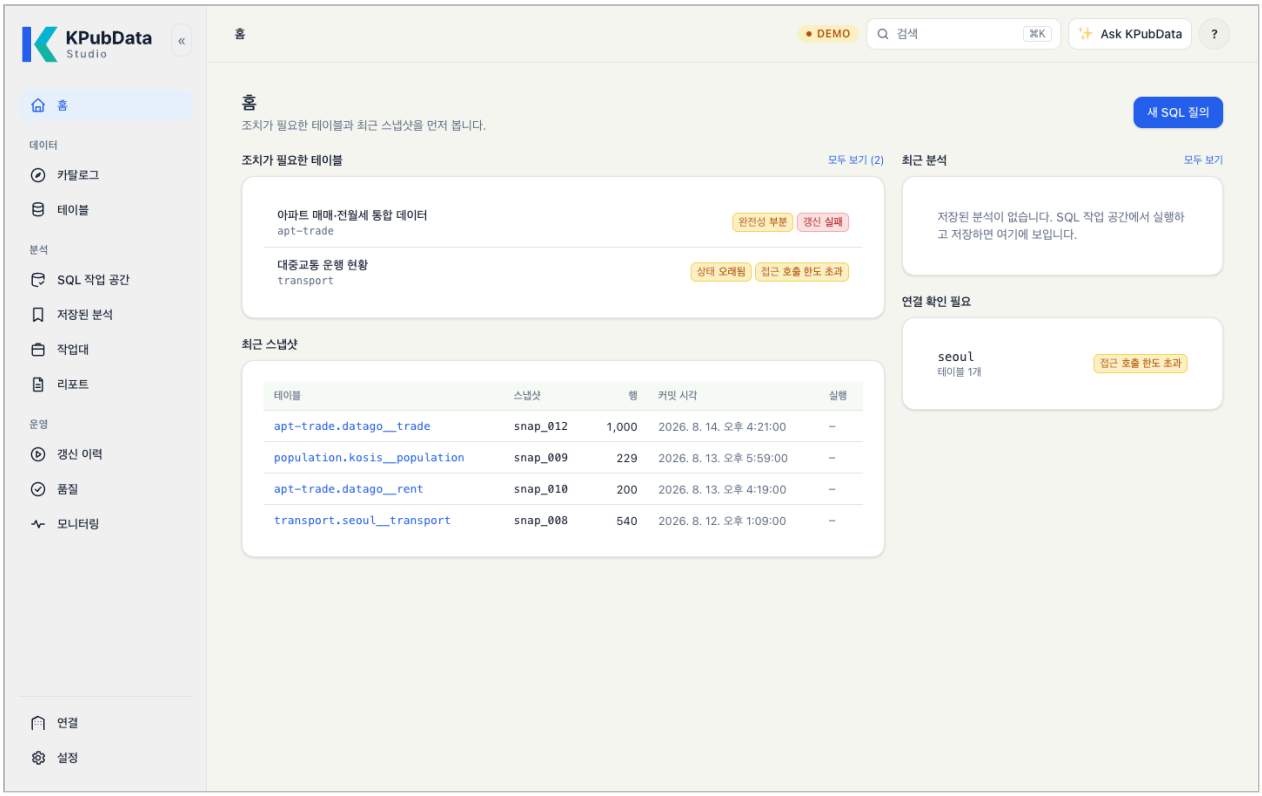
- 테이블 만들기(`/add`)는 메뉴가 아니라 카탈로그·테이블 화면의 동작으로 진입
- 상단 바: 경로 표시, Demo·Beta 배지, 통합 검색(⌘K), Ask KPubData 버튼, 계정 메뉴
- 이전 경로(`/datasets`, `/builds`, `/provider`, `/validate`, `/preview`, `/artifacts`)는 새 경로로 자동 이동

화면 설계서 — 로그인 (1/20)



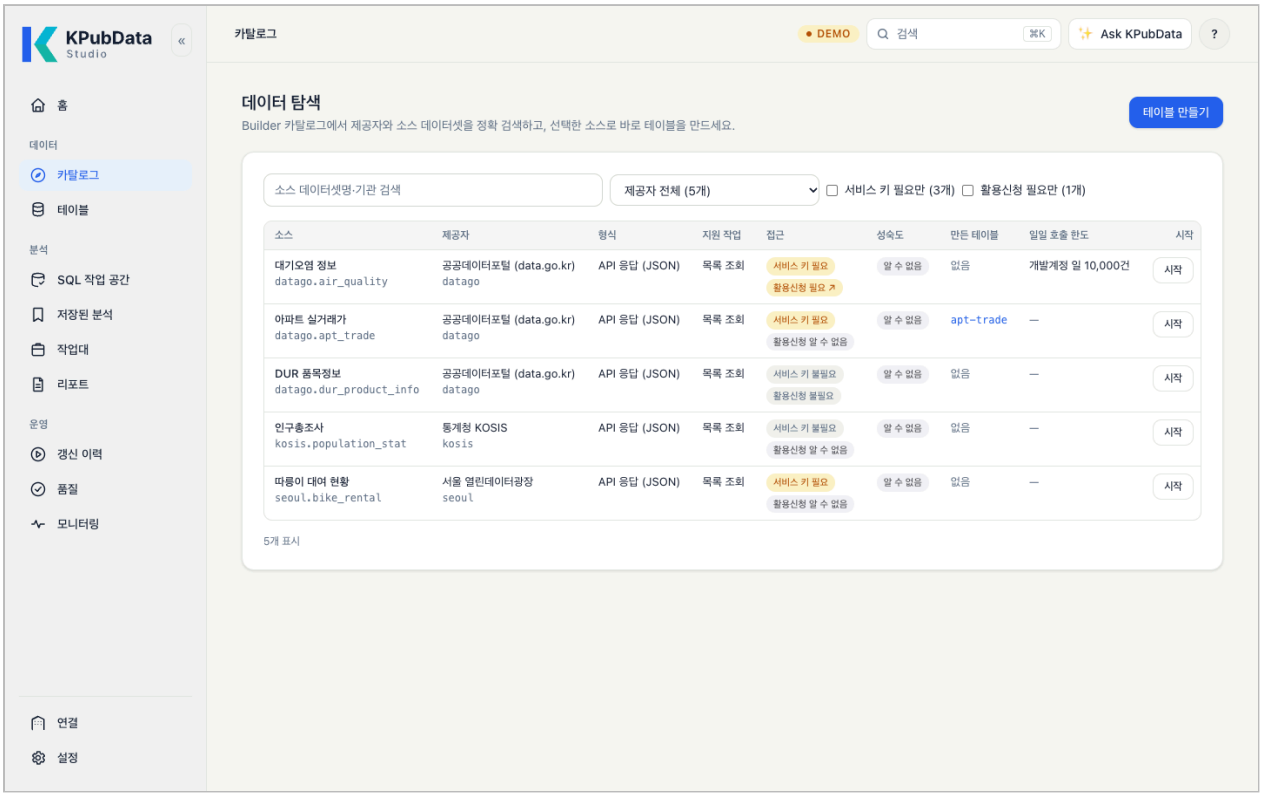
화면 개요	Keycloak 로그인으로 들어가는 진입 화면
주요 UI 구성요소	• 좌측 소개 영역: 제품 문구와 핵심 단계 배지 7종 (Source·BuildSpec·Preview·Validate·Build·Quality·AI) • 우측 로그인 카드: Google 로그인, 이메일 로그인 버튼
상태·상호작용	• 로그인 버튼 클릭 시 Keycloak 인증 화면으로 이동 (OIDC Authorization Code + PKCE) • 데모 모드에서만 이메일·비밀번호 입력 폼 표시 • 인증된 사용자는 원래 가려던 화면으로 이동

화면 설계서 — 홈 (2/20)



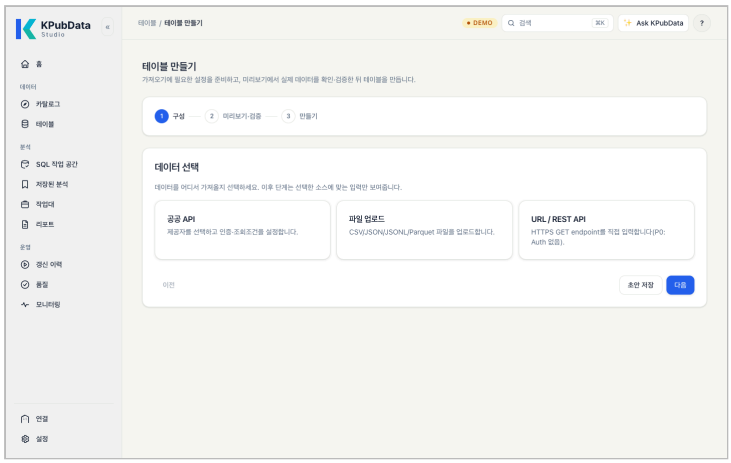
화면 개요	조치가 필요한 테이블과 최근 변경을 먼저 보여주는 시작 화면
주요 UI 구성요소	• 조치가 필요한 테이블 (최대 6개) • 최근 스냅샷과 이를 만든 실행 (최대 5개) • 최근 저장한 분석, 연결 문제 • 상단 '새 SQL 쿼리' 버튼
상태·상호작용	• 테이블과 실행이 하나도 없으면 첫 실행 체크리스트(이용 신청 → 키 등록 → 키 확인 → 첫 테이블) 표시 • 웨어하우스가 없는 배포에서는 최근 스냅샷 대신 최근 실행 표시 • 모든 항목은 한 번의 클릭으로 해당 테이블·분석으로 이동

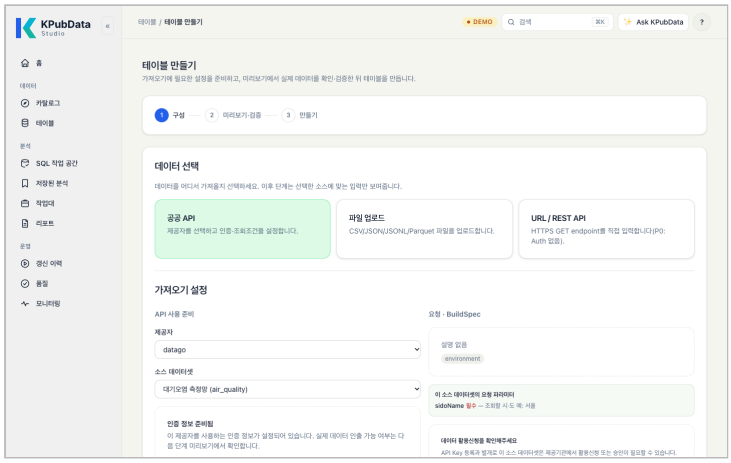
화면 설계서 — 카탈로그 (3/20)



화면 개요	Builder가 제공하는 소스 데이터셋을 비교 표로 찾는 화면
주요 UI 구성요소	• 검색창, 제공 기관 선택, '서비스 키 필요'·'활용 신청 필요' 체크박스 • 비교 표: 소스, 제공 기관, 표현, 지원 동작, 접근 조건, 만든 테이블, 호출 한도, 시작 • 상단 '테이블 만들기' 버튼
상태·상호작용	• 필터 조건은 주소(`?q=&provider=`)에 남아 공유 가능 • 행의 시작 버튼 클릭 시 소스가 채워진 채 테이블 만들기로 이동 • 로딩·오류(다시 시도)·결과 없음 상태를 구분해 표시

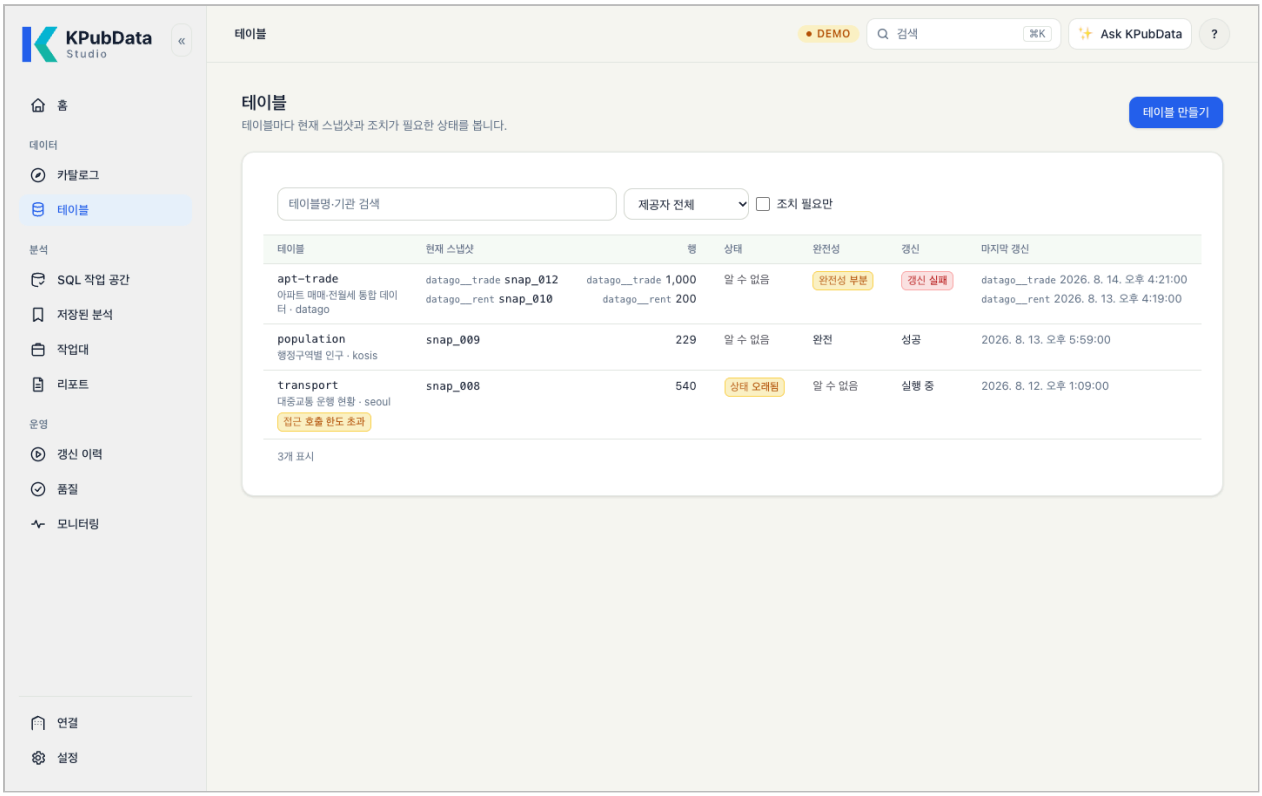
화면 설계서 — 테이블 만들기 (4/20)





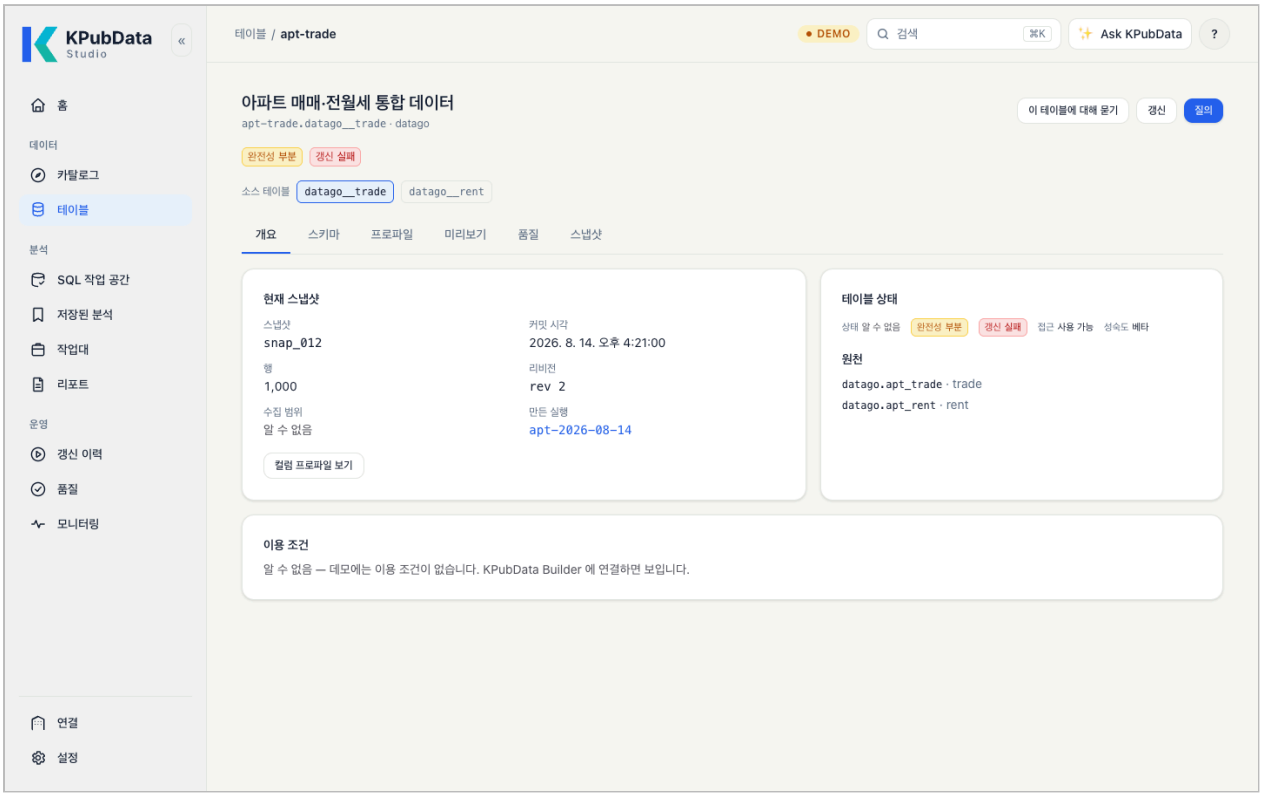
화면 개요	소스 선택부터 생성까지 3단계로 진행하는 마법사
주요 UI 구성요소	• 단계 표시: 구성 → 미리보기·검증 → 만들기 • 1단계: 소스 종류 카드 3종(공공 API, 파일 업로드, URL / REST API)과 가져오기 설정 • 2단계: 소스별 스키마·샘플과 검증 결과 • 3단계: 최종 명세, 테이블 이름, 같은 이름의 테이블 안내 • 하단: 이전, 초안 저장, 다음
상태·상호작용	• 소스 종류를 고르지 않았거나 명세 오류가 있으면 다음 단계 차단 • 자격 증명이 없으면 연결 화면으로 안내 후 복귀 • 미리보기가 오래되면 만들기 차단 • 제출이 수락되면 초안을 정리하고 실행 상세로 이동

화면 설계서 — 테이블 (5/20)



화면 개요	만든 테이블을 현재 스냅샷과 상태로 확인하는 목록 화면
주요 UI 구성요소	• 검색창, 제공 기관 선택, '조치 필요만' 필터 • 표: 테이블, 스냅샷, 행 수, 상태, 완전성, 갱신, 마지막 갱신
상태·상호작용	• 행 클릭 시 테이블 상세로 이동 • 웨어하우스가 없으면 스냅샷 열을 숨기고 안내 한 줄 표시 • 테이블이 없으면 테이블 만들기로 가는 안내 표시

화면 설계서 — 테이블 상세 (6/20)



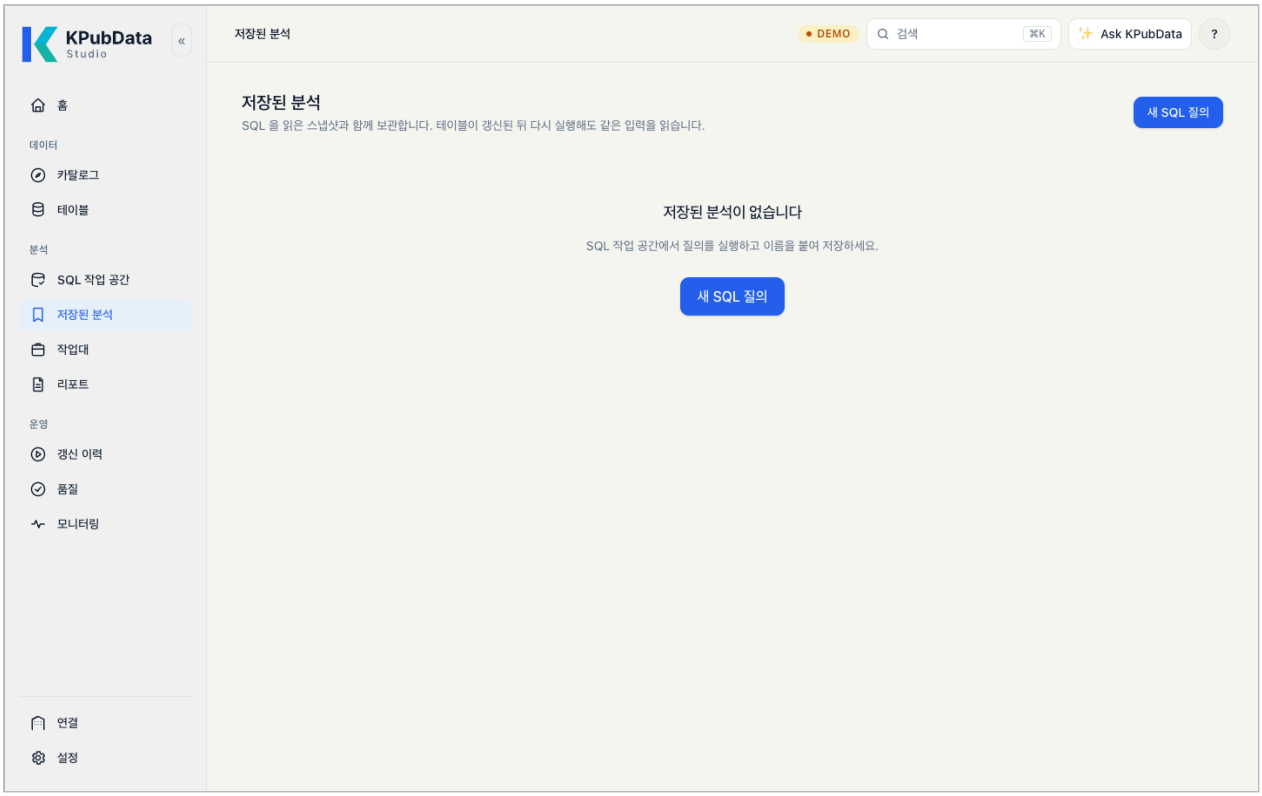
화면 개요	테이블 하나의 스냅샷·스키마·품질을 탭으로 확인하는 화면
주요 UI 구성요소	• 탭: 개요, 스키마, 프로파일, 미리보기, 품질, 스냅샷 • 상단 동작: 이 테이블에 대해 질문, 갱신, 쿼리 • 소스가 여러 개인 테이블은 소스 전환 제공
상태·상호작용	• 과거 스냅샷을 볼 때 '현재로 돌아가기' 버튼 표시 • 스냅샷 탭에서 실행 상세·명세 편집·게시로 이동 • 커밋된 스냅샷이 없으면 실행 기준 보기(개요·스키마·미리보기·품질·실행)로 표시

화면 설계서 — SQL 작업 공간 (7/20)



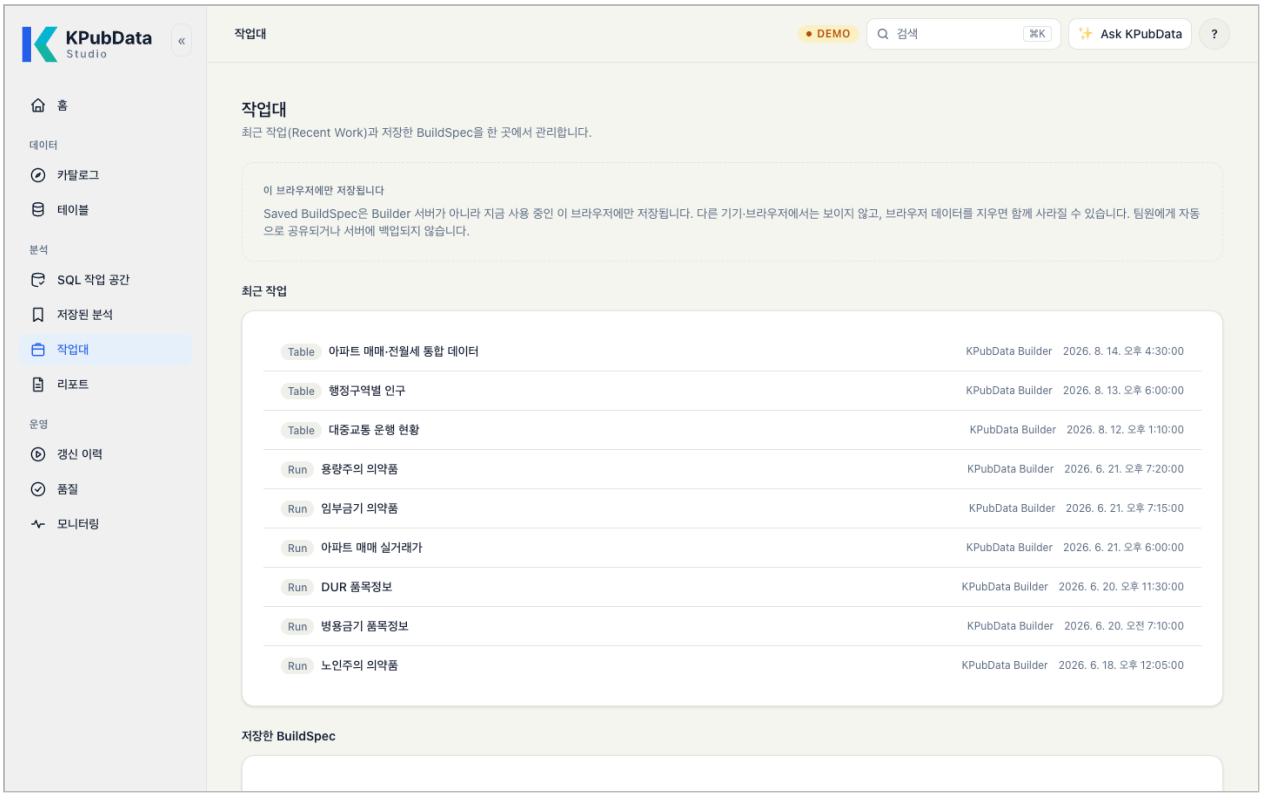
화면 개요	테이블 스냅샷 하나에 읽기 전용 SQL을 실행하는 화면
주요 UI 구성요소	• 테이블 탐색기 (테이블 → 소스 → 컬럼) • 스냅샷 선택(기본값 현재), SQL 입력창, 실행 버튼 • 결과 표(`table@snapshot · rev N`), 분석 저장, 집계 차트, 결과 내보내기
상태·상호작용	• ⌘/Ctrl+Enter로 실행 • 쿼리 하나에 테이블 하나만 조회 • 기본 문장은 `SELECT * FROM dataset LIMIT 100` • SQL 안전성 검사는 Builder가 수행

화면 설계서 — 저장된 분석 (8/20)



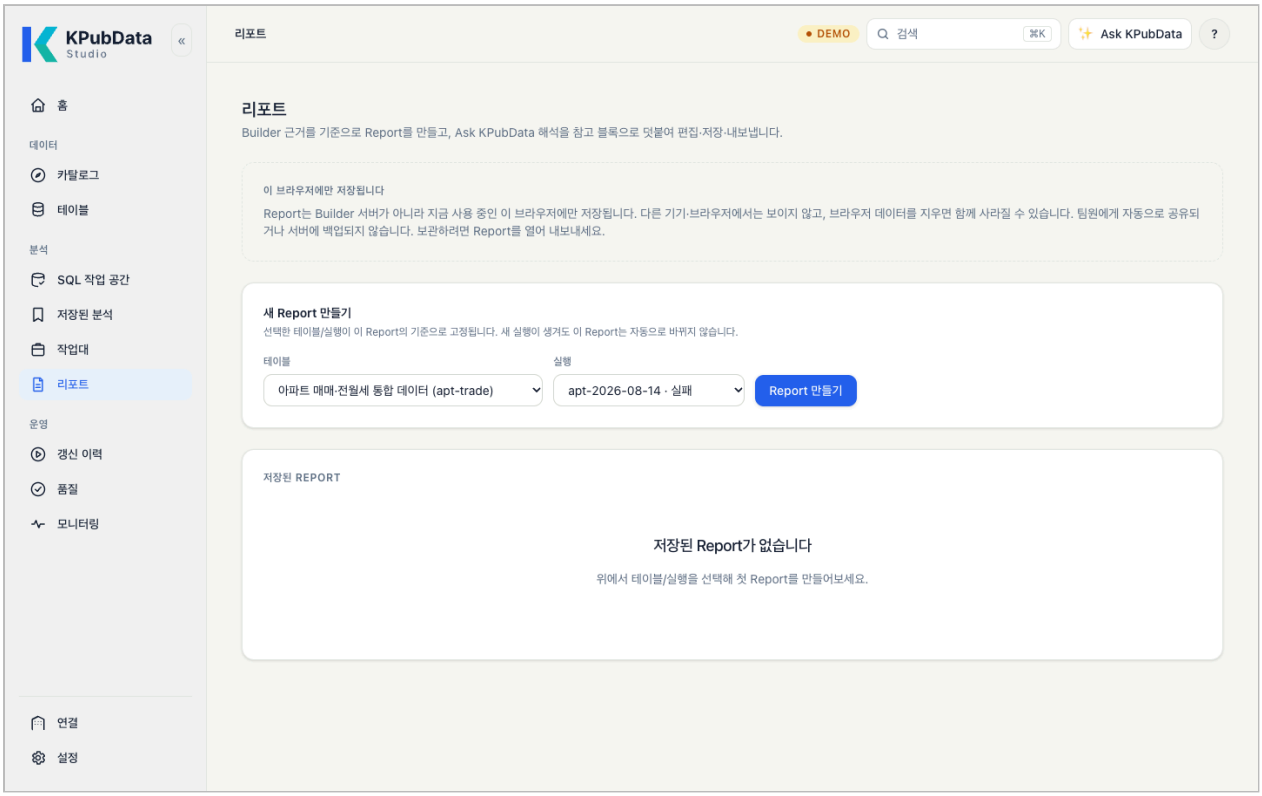
화면 개요	스냅샷에 고정해 저장한 분석을 다시 실행하는 화면
주요 UI 구성요소	• 분석 카드: 이름, `table@snapshot`, 저장 정보 • 버튼: 다시 실행, SQL 작업 공간에서 열기, 삭제
상태·상호작용	• 저장된 분석이 없으면 SQL 작업 공간으로 가는 안내 표시 • 삭제는 확인 후 진행

화면 설계서 — 작업대 (9/20)



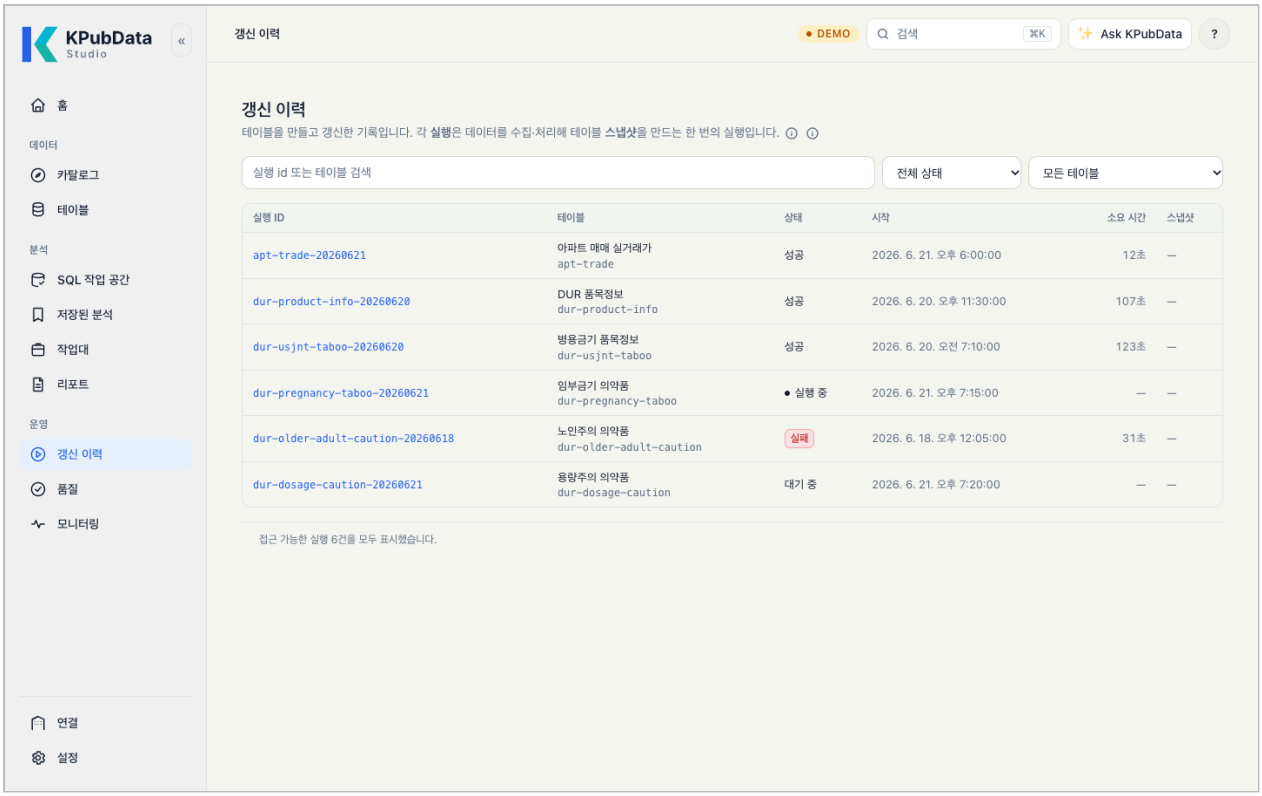
화면 개요	최근 작업과 저장한 BuildSpec을 한곳에서 관리하는 화면
주요 UI 구성요소	• 최근 작업 목록 (테이블·실행·리포트·저장한 명세) • 저장한 BuildSpec: 이름 변경, 복제, 삭제
상태·상호작용	• 저장한 BuildSpec은 이 브라우저에만 보관됨을 배지로 표시 • 저장한 명세를 열면 테이블 만들기로 이동

화면 설계서 — 리포트 (10/20)



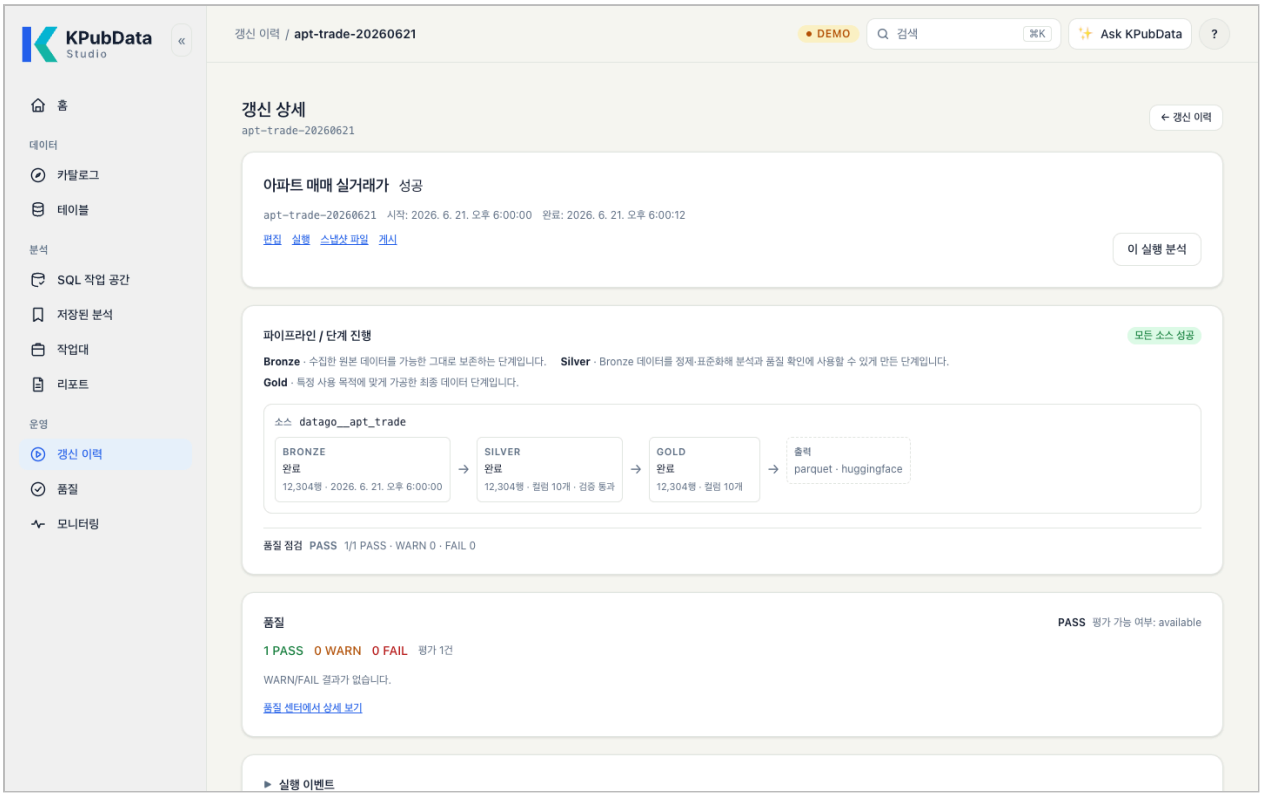
화면 개요	실행 근거를 기준으로 리포트를 만들고 편집하는 화면
주요 UI 구성요소	• 새 리포트 만들기: 테이블·실행 선택 • 리포트 목록: 열기, 이름 변경, 복제, 삭제 • 편집기: 근거 영역(1~6), AI 분석(7), 사용자 메모(8), 자동 저장
상태·상호작용	• 리포트는 이 브라우저에만 저장됨을 배지로 표시 • 근거 새로 고침, Markdown·HTML 내려받기, 인쇄 제공 • 선택한 실행이 기준으로 고정되며 새 실행이 생겨도 자동으로 바뀌지 않음

화면 설계서 — 갱신 이력 (11/20)



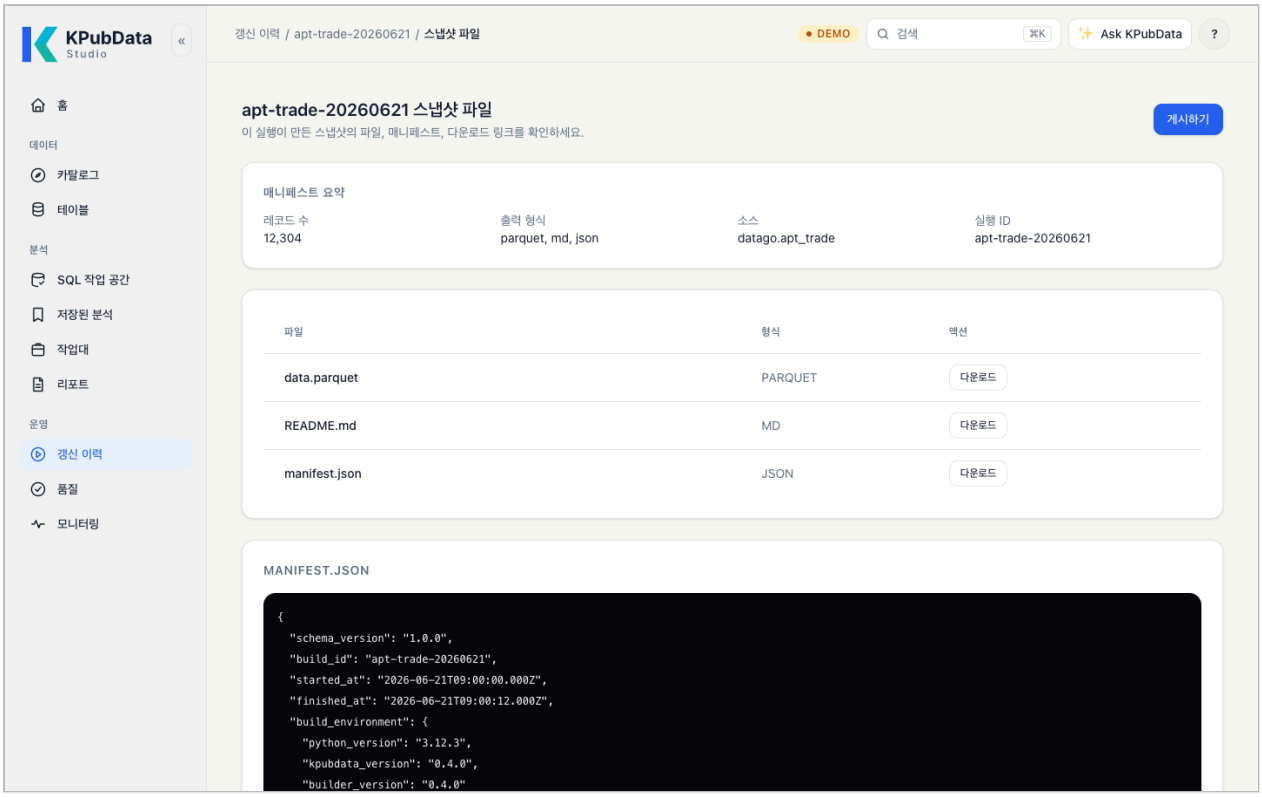
화면 개요	테이블을 만들고 갱신한 실행을 한 표로 조회하는 화면
주요 UI 구성요소	• 검색창, 상태 필터(전체·성공·실패·실행 중·대기·취소), 테이블 필터 • 표: 실행 ID, 테이블, 상태, 시작, 소요 시간, 스냅샷
상태·상호작용	• 행 클릭 시 실행 상세로 이동 • '더 보기'로 이전 실행 추가 조회 • 필터 조건은 주소에 남음

화면 설계서 — 실행 상세 (12/20)



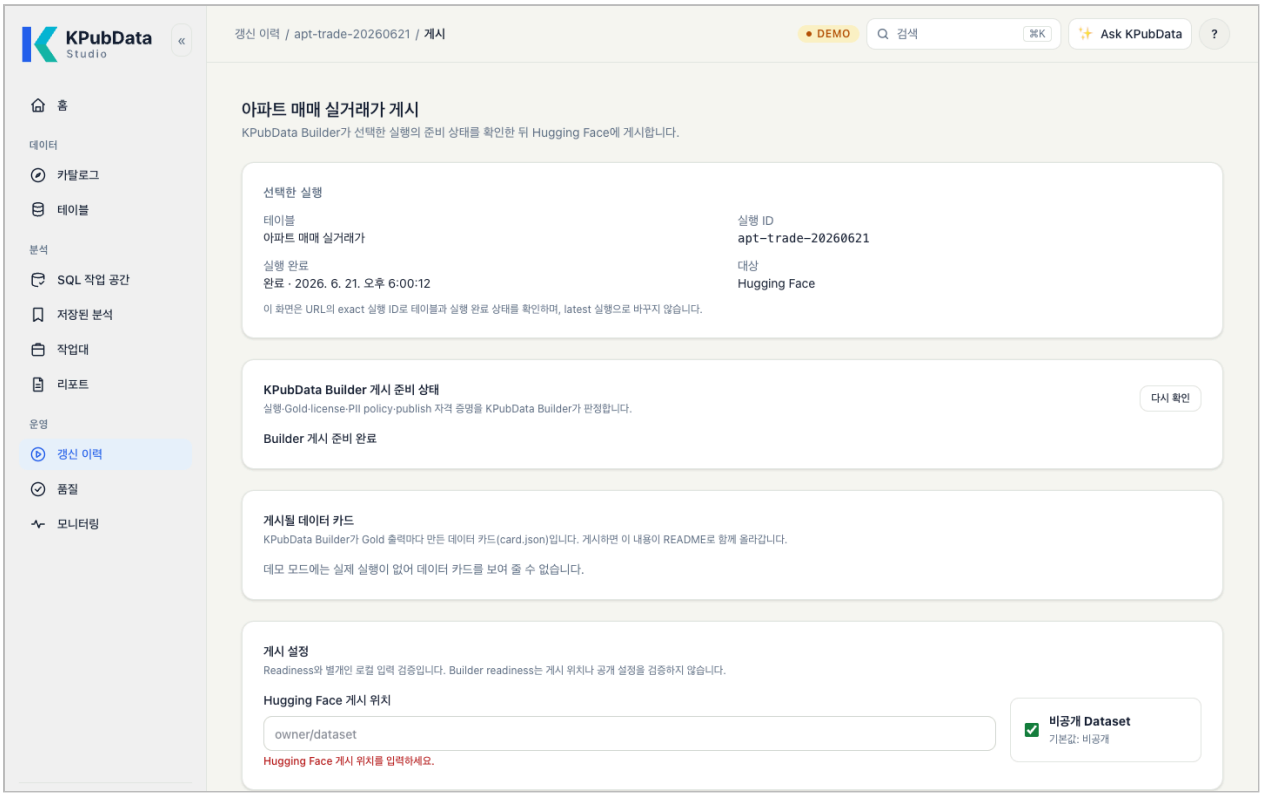
화면 개요	실행 하나의 단계별 결과와 품질, 이벤트를 확인하는 화면
주요 UI 구성요소	• 상단: 제목, 상태, 취소 버튼(진행 중일 때), 편집·산출물·게시 링크 • 소스별 진행: Bronze → Silver → Gold → 출력 • 품질 결과, 실패 근거, 실행 이벤트 타임라인, 명세 스냅샷
상태·상호작용	• 진행 중에는 800ms 간격으로 상태 갱신, 이벤트는 3초 간격 • 취소는 단계 경계에서 반영되며 확인 후 진행 • 실행 없음·조회 실패·권한 없음을 각각 다른 안내로 표시

화면 설계서 — 산출물 (13/20)



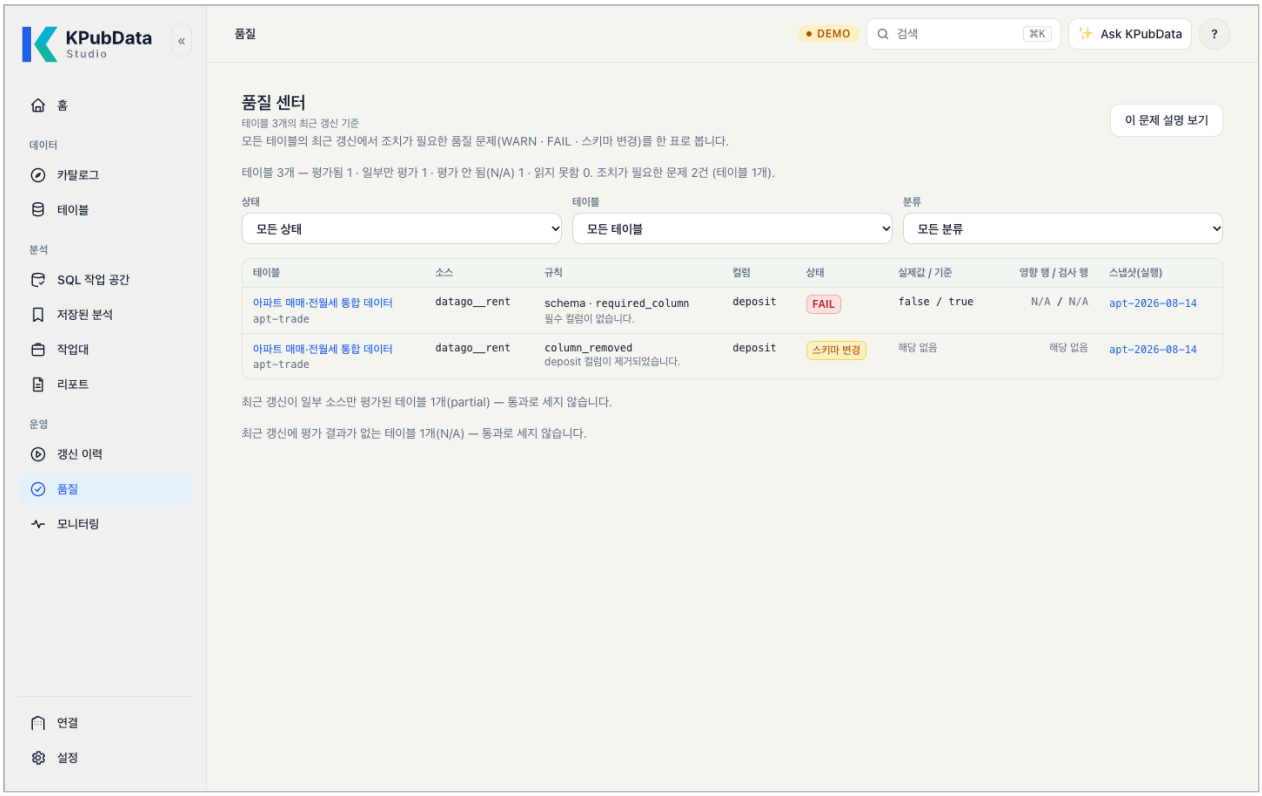
화면 개요	실행이 만든 파일과 Manifest를 확인하는 화면
주요 UI 구성요소	• Manifest 요약, 내려받을 수 있는 파일 목록, manifest.json 원문
상태·상호작용	• 영역별로 따로 불러와 한 영역의 실패가 다른 영역을 막지 않음 • 파일 내려받기는 게시와 구분됨

화면 설계서 — 게시 (14/20)



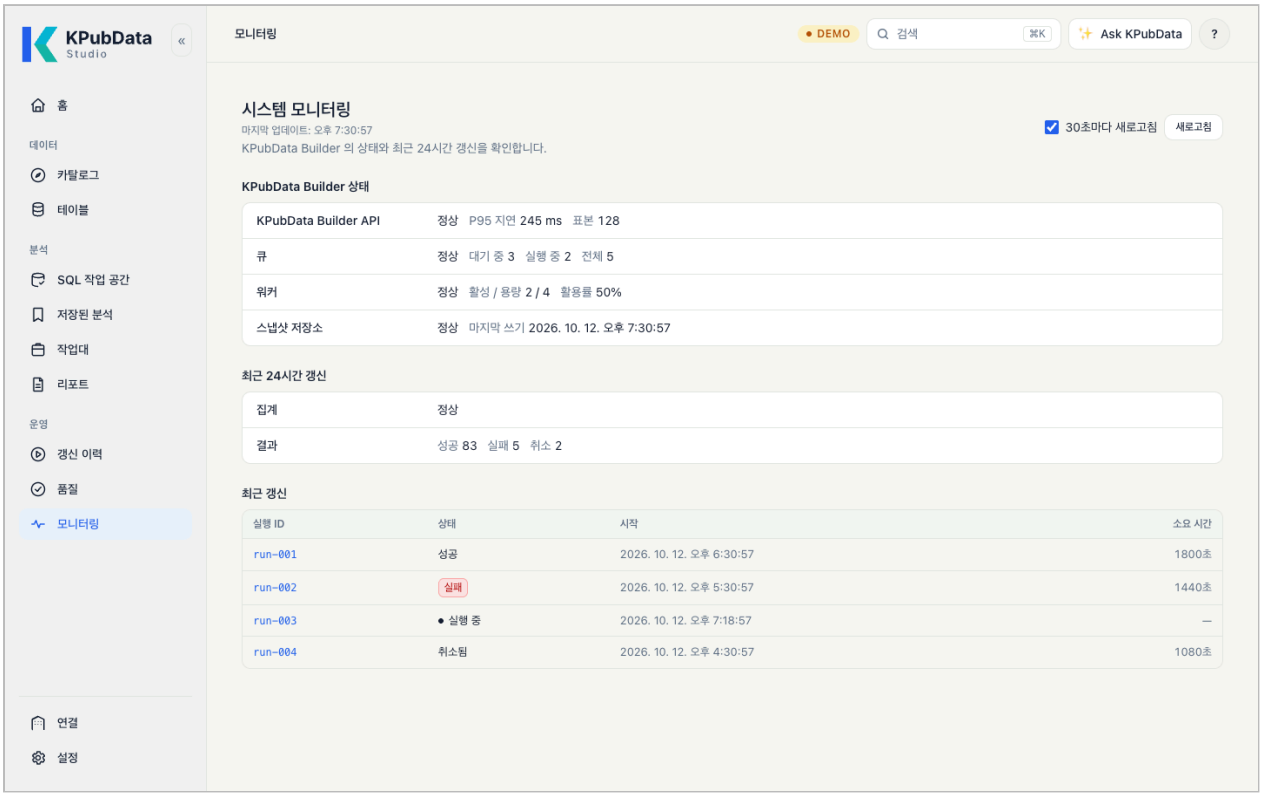
화면 개요	실행 결과를 Hugging Face에 게시하는 화면
주요 UI 구성요소	• 게시 준비 상태 (차단 사유·경고) • 데이터 카드 미리보기 • 대상 저장소(`owner/dataset`), 비공개 여부 • 검토·확인 후 '지금 게시'
상태·상호작용	• 차단 사유가 있으면 게시 버튼 비활성 • 비상업 조건 데이터는 별도 확인 절차 • 결과가 불명확하면 자동 재시도 없이 복구 패널(대조·영수증 초기화) 제공

화면 설계서 — 품질 (15/20)



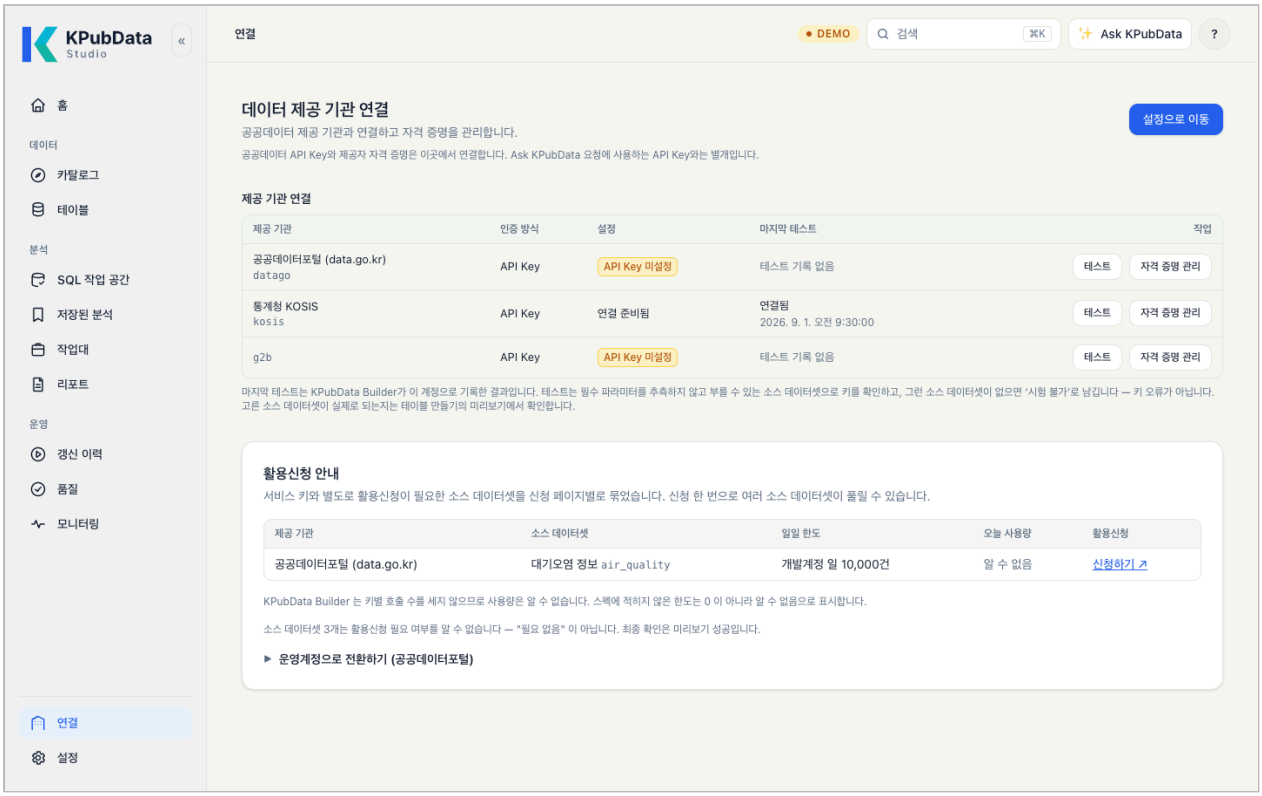
화면 개요	테이블 전체의 품질 경고와 스키마 변화를 모아 보는 화면
주요 UI 구성요소	• 필터: 상태, 테이블, 분류 • 표: 테이블, 소스, 규칙, 컬럼, 상태, 실제값 / 기준값, 영향 행 수, 스냅샷
상태·상호작용	• 점수 대신 검사 결과(PASS·WARN·FAIL·N/A)만 표시 • 각 행은 테이블 상세의 품질 탭으로 연결 • '도우미로 분석' 버튼 제공

화면 설계서 — 모니터링 (16/20)



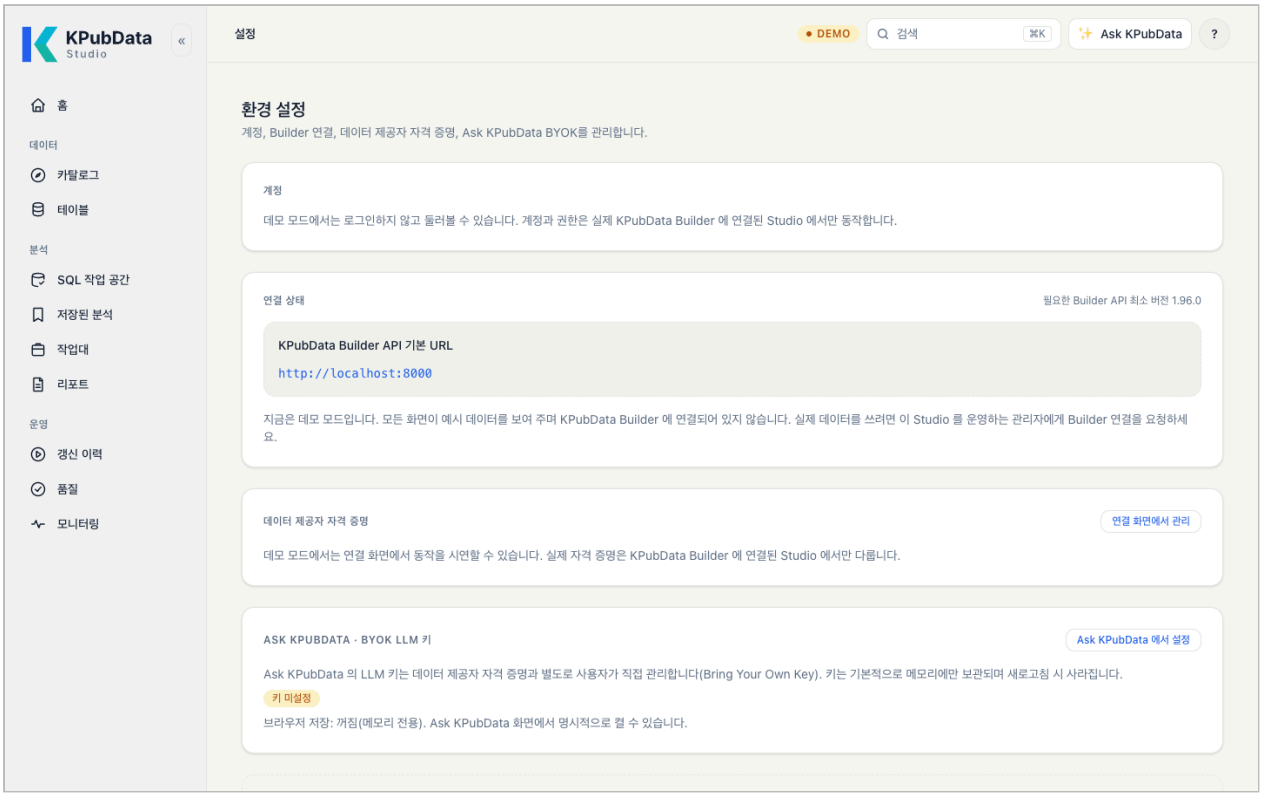
화면 개요	Builder 상태와 최근 24시간 실행 현황을 확인하는 화면
주요 UI 구성요소	• Builder 상태: API, 대기열, 작업자, 스냅샷 저장소 • 최근 24시간 실행 통계, 최근 갱신 목록 • 자동 새로 고침 체크박스, 수동 새로 고침 버튼
상태·상호작용	• 30초 간격으로 조회하며 탭이 보이지 않으면 중단 • 권한이 없으면 안내 한 줄만 표시 • 일부 값만 받지 못하면 저하 상태 배너 표시

화면 설계서 — 연결 (17/20)



화면 개요	공공데이터 제공 기관의 자격 증명을 관리하는 화면
주요 UI 구성요소	• 표: 제공 기관, 인증 방식, 설정 여부, 마지막 확인, 동작 • 선택한 기관의 자격 증명 패널, 키 확인, 활용 신청 안내
상태·상호작용	• 단일 사용자 배포: 키를 저장·삭제 • 다중 사용자 배포: 키를 이번 세션에만 사용 (메모리 보관) • 키 원문은 화면에 표시하지 않음

화면 설계서 — 설정 (18/20)



화면 개요	Builder 연결과 계정, AI 키 상태를 확인하는 화면
주요 UI 구성요소	• 연결: Builder 주소와 계약 버전 확인 • 개인정보 안내, 계정, 제공 기관 자격 증명 요약 • Ask KPubData 키(BYOK) 상태
상태·상호작용	• Builder 계약 버전이 최소 요건(1.96.0)보다 낮으면 모든 화면에 경고 배너 • AI 키는 기본적으로 메모리에만 보관

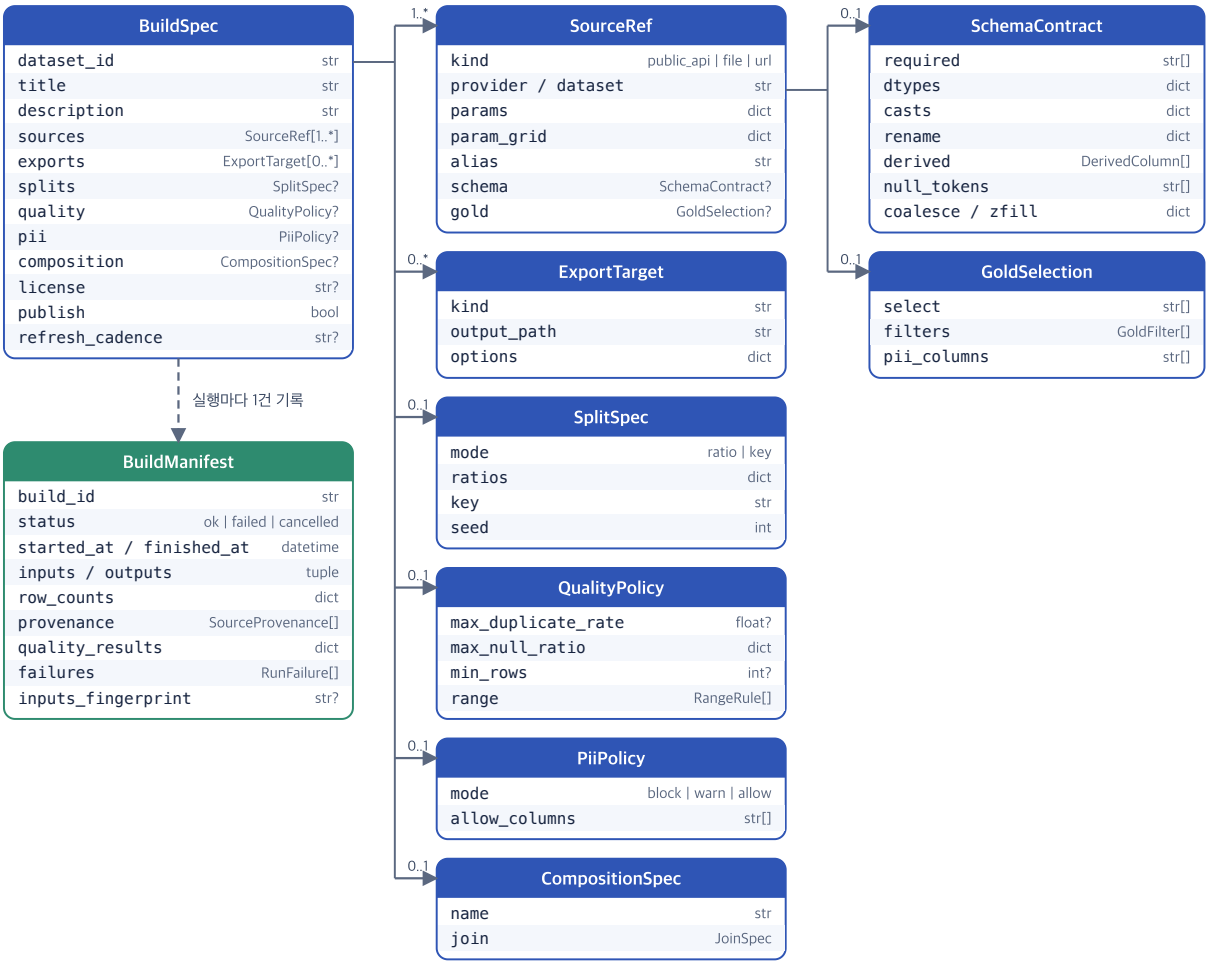
화면 설계서 — Ask KPubData (19/20)

화면 개요	현재 화면의 테이블·실행·품질 문맥으로 질문하고 다음 동작까지 이어 가는 AI 도우미
주요 UI 구성요소	• 문맥 표시(테이블·실행·품질·단계), 추천 질문, 관련 데이터셋 • 답변, 근거 목록, 생성된 SQL(실행·복사), 제안 동작(승인·거절) • 키 입력 폼: API 키, 모델, 주소
상태·상호작용	• 오른쪽 서랍으로 열리며 전체 화면(`/assistant`)도 제공 • SQL은 사용자가 실행을 눌렀을 때만 Builder로 전달 • 문맥이 바뀐 뒤의 제안 동작은 실행할 수 없음을 표시

화면 설계서 — 관리 (20/20)

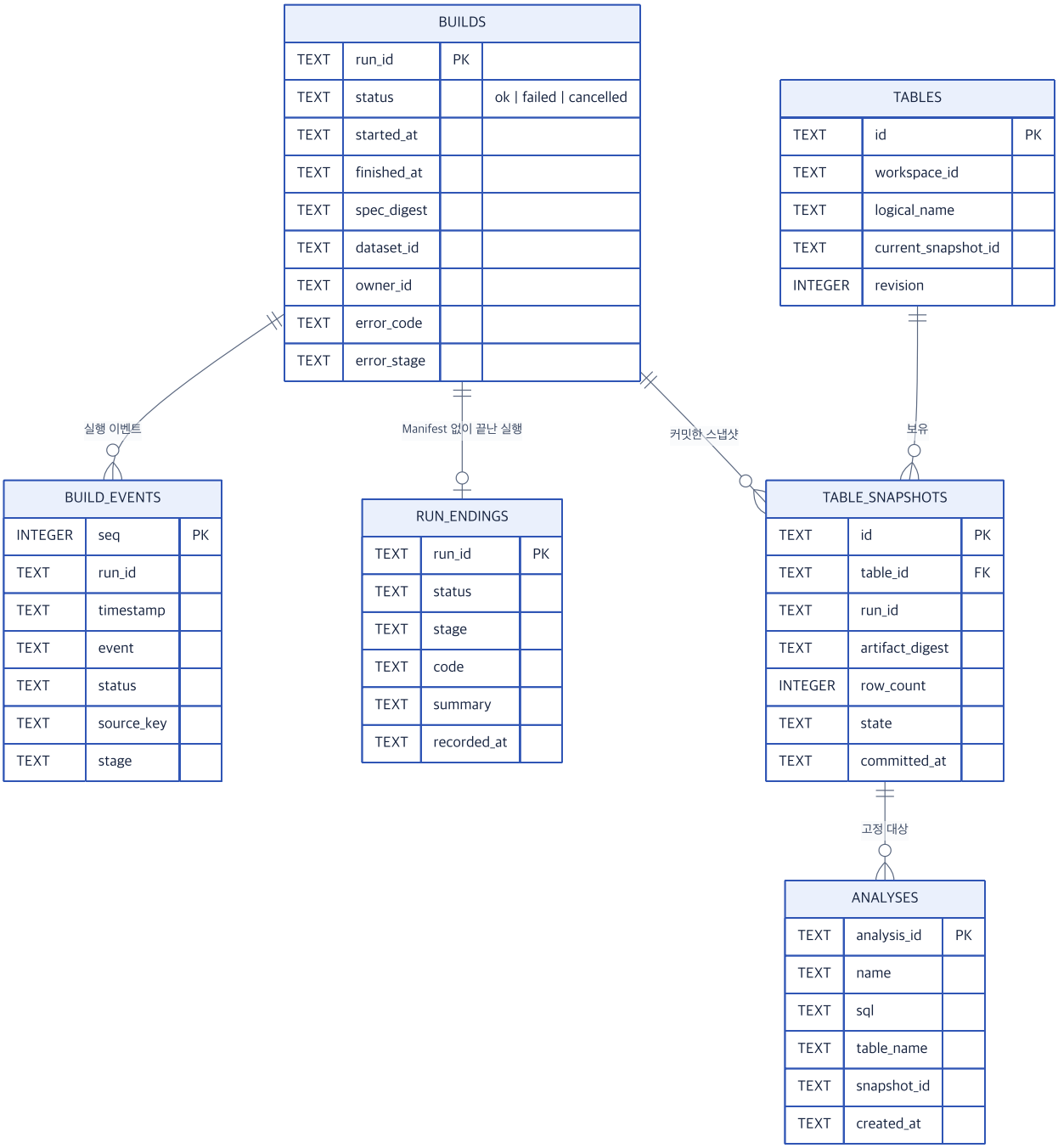
화면 개요	관리자가 운영 정책과 전체 실행, 가입 신청을 확인하는 화면
주요 UI 구성요소	• 현재 적용 중인 정책 • 전체 사용자의 실행 목록 (메타데이터만) • 가입 신청 목록: 승인, 거절
상태·상호작용	• 관리자에게만 메뉴 표시 • 카드별로 로딩·권한 없음·미지원·실패 상태를 따로 표시

엔티티 관계도 - 핵심 도메인 모델



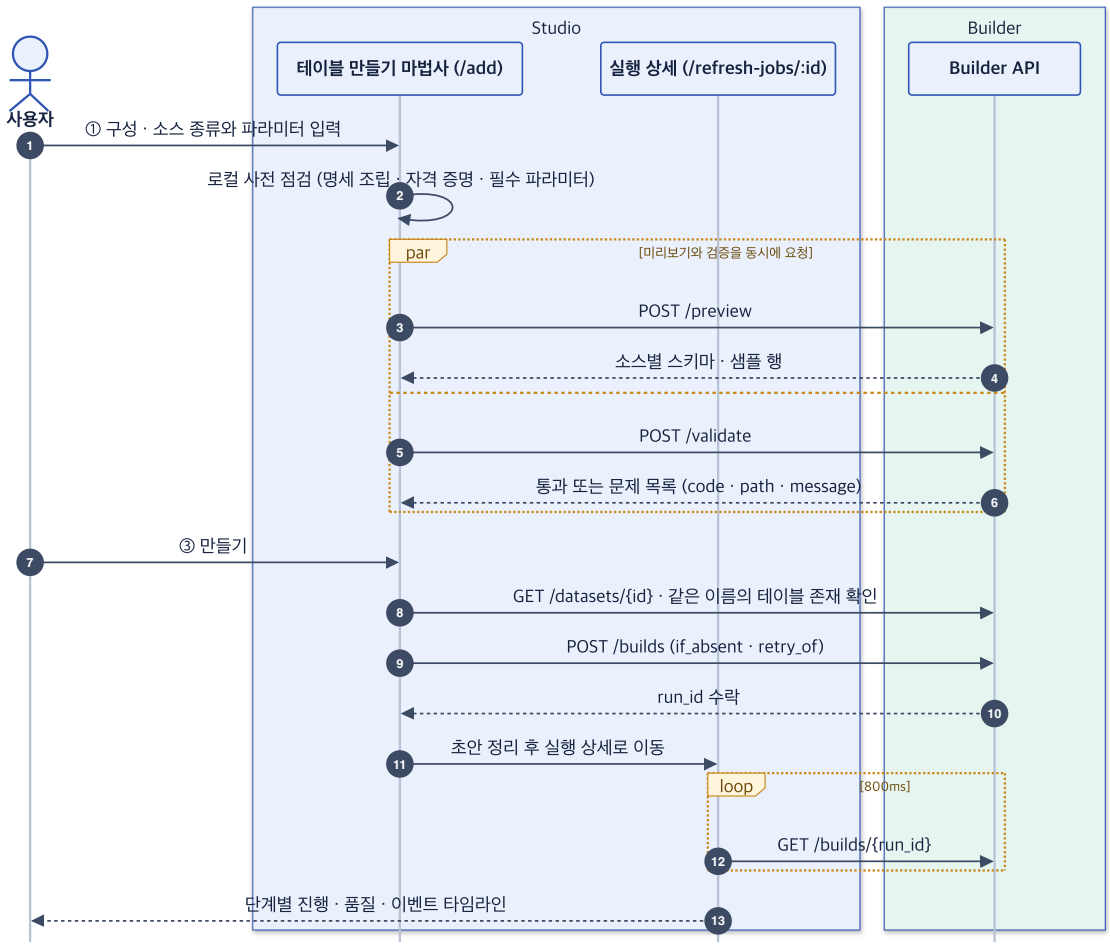
- BuildSpec은 실행 입력을 선언하는 계약이며 Python dataclass로 정의함
- BuildManifest는 실행마다 한 건씩 남는 결과 기록임
- 관계형 스키마가 아니므로 UML 클래스 다이어그램 표기법으로 정리함

엔티티 관계도 - 저장 구조 (ERD)



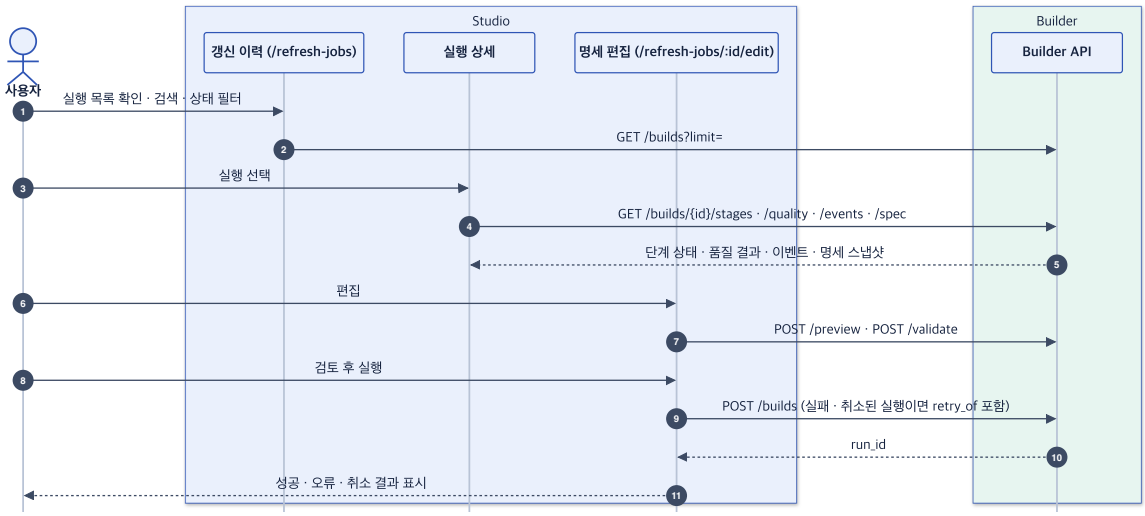
- 상태는 SQLite와 로컬 파일에만 보관함
- `builds`는 manifest.json과 종료 기록에서 다시 만들 수 있는 파생 인덱스임
- `tables`·`table_snapshots`는 웨어하우스 카탈로그이며 스냅샷 본문은 Parquet 파일임
- 주요 컬럼만 표시함

기능 처리도 - 테이블 만들기



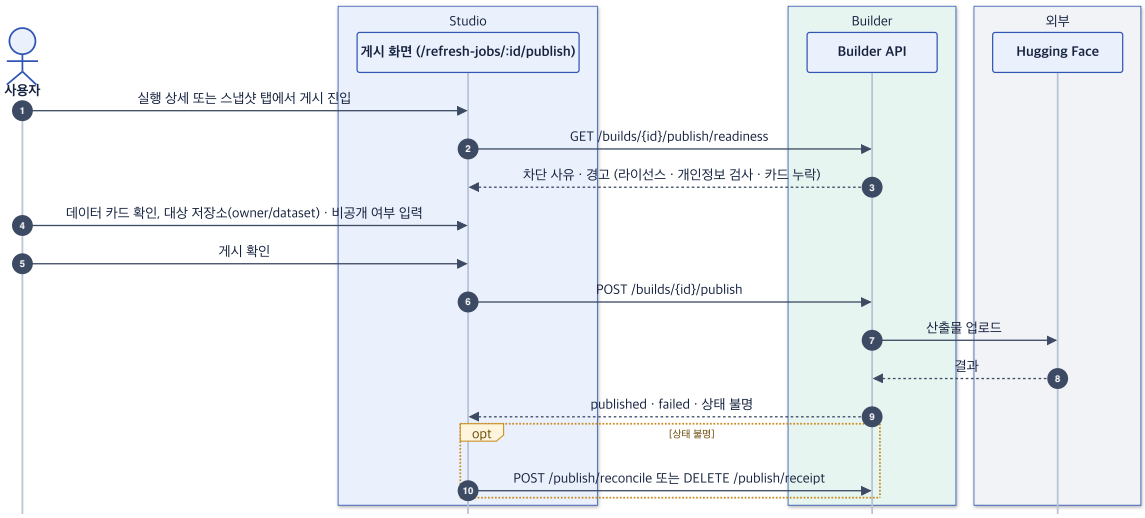
- 사용자 조작(①⑦)은 Studio 안에서 처리되고, Builder API 호출은 미리보기·검증·존재 확인·제출 네 곳임
- 미리보기와 검증은 동시에 요청하며, 늦게 도착한 이전 응답은 버림
- 제출 직전에 같은 이름의 테이블이 있는지 다시 확인하고, 결과가 달라졌으면 제출하지 않음
- 마법사는 완료를 기다리지 않고 실행 상세로 넘어가며, 진행은 실행 상세가 따라감

기능 처리도 - 실행 검토·재실행



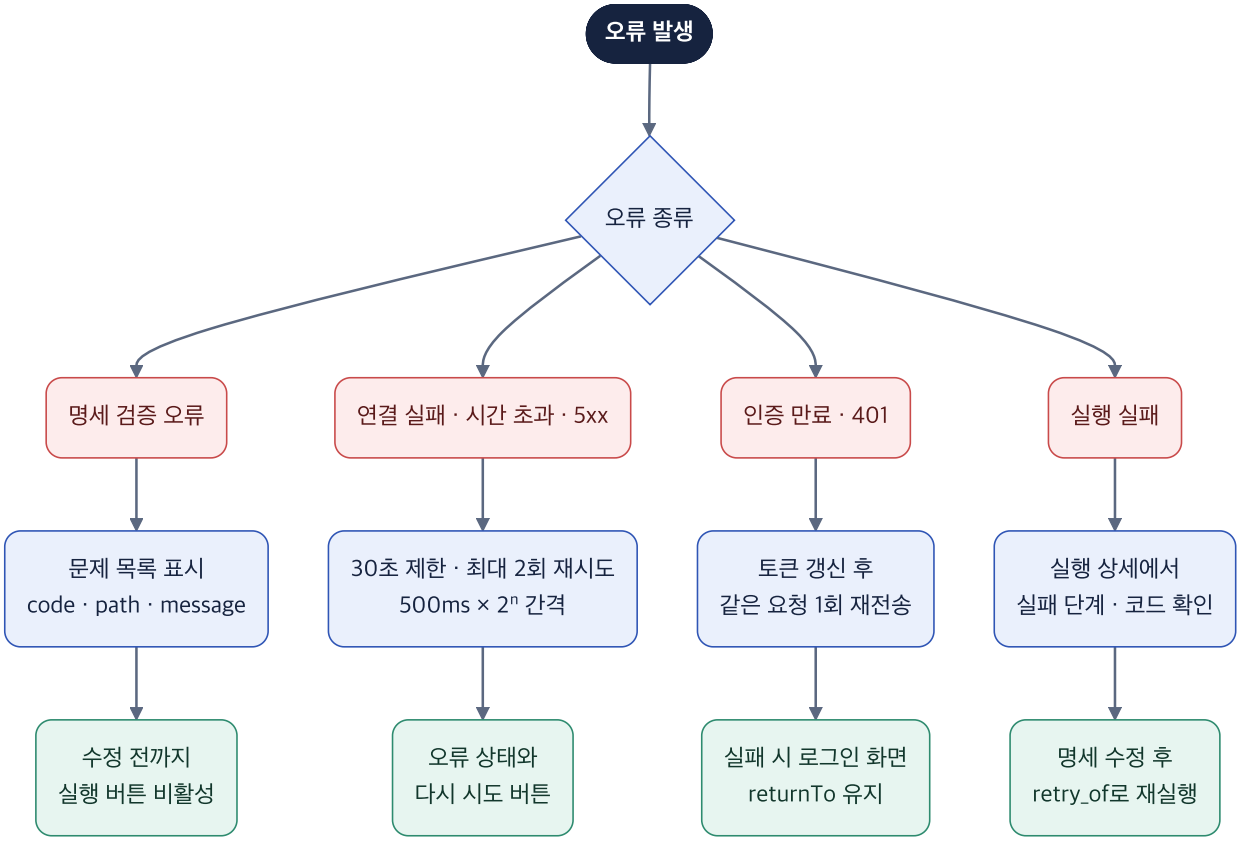
- 실행 상세는 단계·품질·이벤트·명세를 따로 조회해 한 영역의 실패가 전체를 막지 않게 함
- 실패하거나 취소된 실행을 다시 돌릴 때만 `retry_of`를 보내며, 수집 체크포인트를 이어받음
- 성공한 실행을 다시 돌리는 것은 갱신이며 새 스냅샷을 만듦

기능 처리도 - 게시



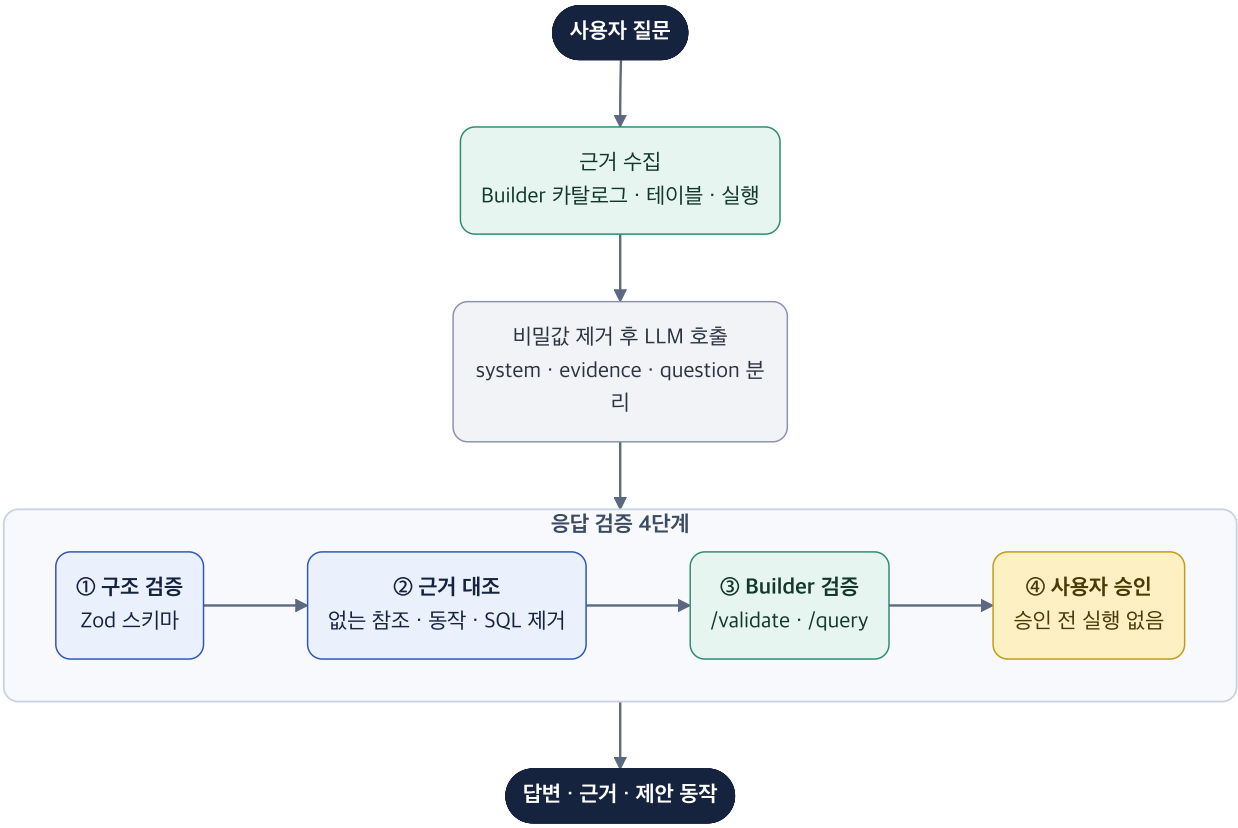
- 게시 전 준비 상태를 조회해 라이선스, 개인정보 검사, 카드 누락 등 차단 사유를 먼저 표시
- Studio의 게시 대상은 Hugging Face이며, Builder CLI는 로컬·Kaggle 게시도 지원
- 게시 요청은 자동 재시도하지 않으며, 결과가 불명확하면 대조 또는 영수증 초기화로 복구

기능 처리도 - 오류 처리



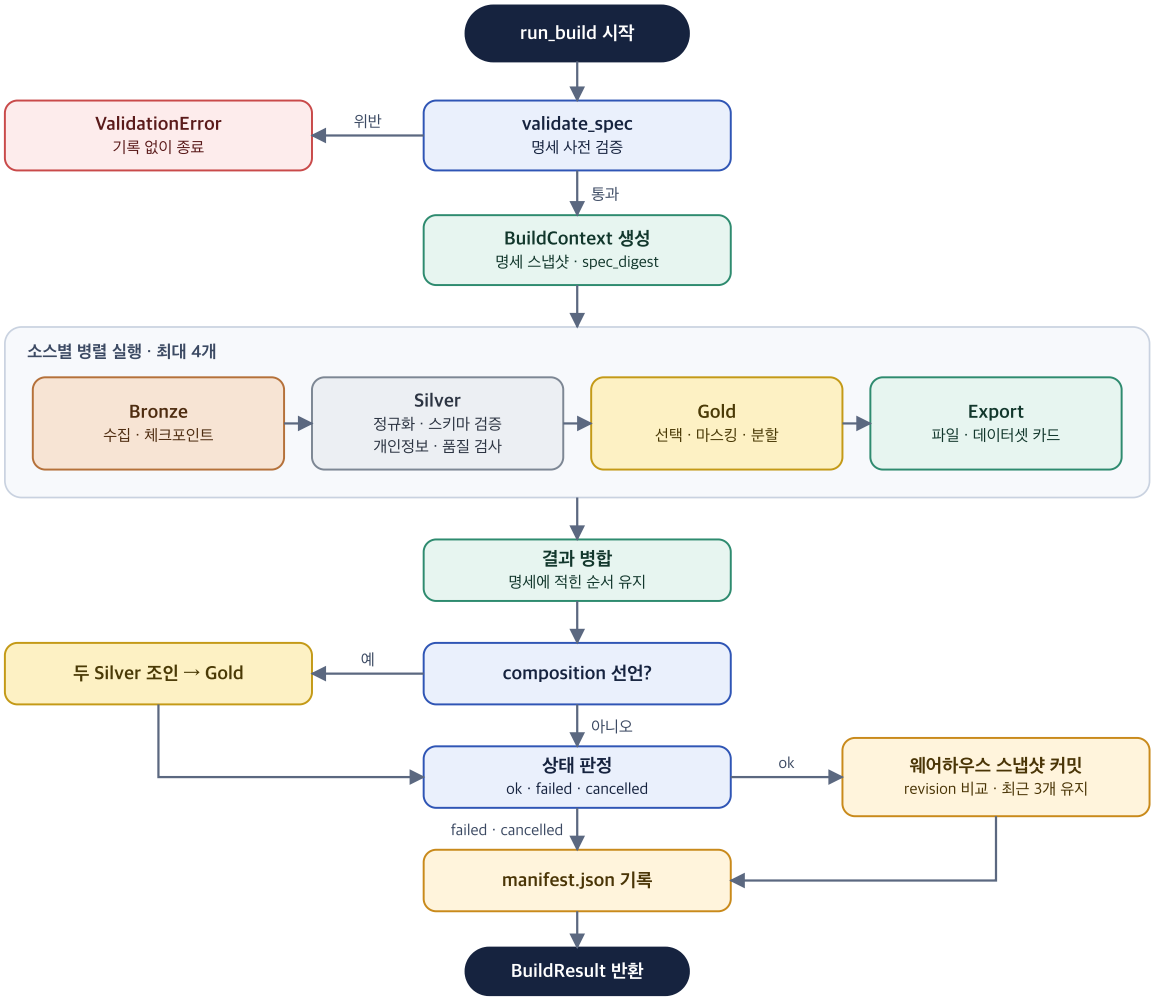
- ① 명세 검증 오류 — 문제 목록을 표시하고 수정 전까지 실행을 막음
- ② 연결 실패 — 30초 제한과 최대 2회 재시도 후 오류 상태와 다시 시도 버튼 표시. 미리보기·제출·게시·쿼리는 재시도하지 않음
- ③ 인증 만료 — 토큰을 갱신해 같은 요청을 한 번 다시 보내고, 실패하면 로그인 화면으로 이동
- ④ 실행 실패 — 실행 상세에서 실패 단계와 코드를 확인하고 명세를 고쳐 재실행
- 오프라인 모드나 자동 재연결은 두지 않음. 주기적 조회가 돌아오면 화면이 갱신됨

기능 처리도 - Ask KPubData 응답 처리



- 프롬프트 역할 분리: 고정 지시(system), 신뢰하지 않는 근거(evidence), 사용자 질문을 구분해 전달
- 전송 전 비밀값 제거: 키·토큰 이름의 필드, `key=value` 형태, 무작위성이 높은 긴 문자열을 자리표시자로 치환
- 4단계 검증: ① Zod 구조 검증 → ② 근거 대조 → ③ Builder 검증 → ④ 사용자 승인
- SQL 안전성 검사는 Studio가 아니라 Builder의 읽기 전용 검사가 담당
- 허용 동작 6종: OPEN_PROVIDER, OPEN_BUILD, OPEN_QUALITY, PATCH_BUILDSPEC, CREATE_BUILD_DRAFT, ADD_REPORT_BLOCK
- 실행 시작, 게시, 자격 증명 변경, SQL 자동 실행은 제안할 수 없음
- LLM 키는 사용자가 직접 입력(BYOK)하며 기본적으로 메모리에만 보관

알고리즘 명세서


run_build 시작
validate_spec
명세 사전 검증
위반
ValidationError
기록 없이 종료
통과
BuildContext 생성
명세 스냅샷 · spec_digest
소스별 병렬 실행 · 최대 4개
Bronze
수집 · 체크포인트
Silver
정규화 · 스키마 검증
개인정보 · 품질 검사
Gold
선택 · 마스킹 · 분할
Export
파일 · 데이터셋 카드
결과 병합
명세에 적힌 순서 유지
composition 선언?
예
두 Silver 조인 → Gold
아니오
상태 판정
ok · failed · cancelled
ok
웨어하우스 스냅샷 커밋
revision 비교 · 최근 3개 유지
failed · cancelled
manifest.json 기록
BuildResult 반환

알고리즘 명세서

- ① `run_build()`가 BuildSpec을 받아 `validate_spec()`으로 먼저 검증. 위반이 있으면 `ValidationError`로 끝나며 작업공간도 기록도 만들지 않음
- ② `BuildContext.create()`로 실행 작업공간을 만들고 명세 스냅샷(buildspec.yaml)과 `spec_digest` 기록
- ③ 소스를 스레드 풀로 병렬 실행 (최대 4개). 결과는 명세에 적힌 순서대로 모음
- ④ 소스마다 Bronze → Silver → Gold → Export 순으로 진행. 단계 경계마다 취소 여부 확인
- ⑤ Silver에서 스키마 검증, 개인정보 검사, 품질 검사를 통과해야 Gold로 넘어감
- ⑥ composition을 선언했으면 두 소스의 Silver를 조인해 하나의 Gold 생성
- ⑦ 상태 판정: 취소되었으면 cancelled, 오류가 하나라도 있으면 failed, 아니면 ok
- ⑧ 상태가 ok일 때만 Gold를 웨어하우스 스냅샷으로 커밋. 시작 시점의 revision과 비교해 충돌하면 거부
- ⑨ manifest.json을 기록하고 `BuildResult(status, outcomes, manifest_path, …)` 반환

상태값

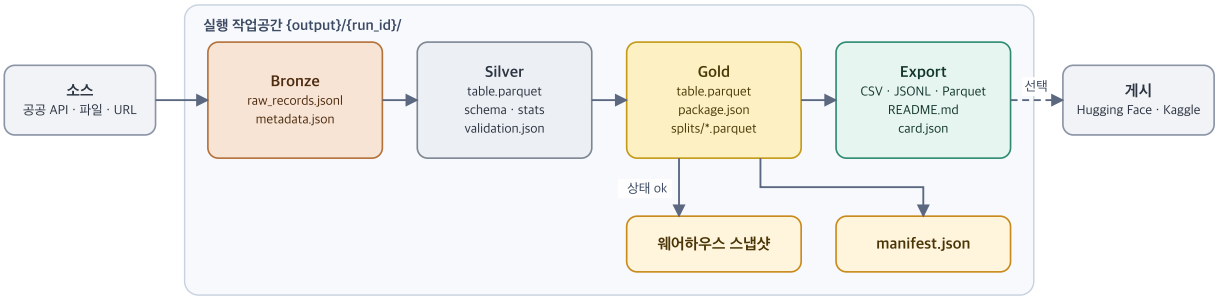
계층	값	비고
오케스트레이터·Manifest	`ok` · `failed` · `cancelled`	실행 결과의 원본 값
HTTP API 작업 상태	`queued` · `running` · `cancelling` · `succeeded` · `failed` · `cancelled`	`ok`는 `succeeded`로 표시. 테이블 커밋이 실패하면 `failed`
품질 검사	`pass` · `warn` · `fail`	평가하지 못한 규칙은 결과에서 제외

Manifest가 남지 않는 경우

- 명세 검증에서 거부된 요청: 실행 자체가 시작되지 않음
- 대기 중 취소, 서버 재시작 등으로 파이프라인에 들어가기 전에 끝난 비동기 실행: `run_endings` 표에 종료 기록으로 남김
- 파이프라인에 들어간 뒤 취소된 실행은 `status=cancelled`, `partial=true`로 Manifest를 남김

데이터 수집처리 정의서

○ Medallion 구조 기반 수집·처리 — Bronze / Silver / Gold 3단계 + Export



단계	처리 내용	사용 기술	입력 → 출력
Bronze (원천 수집)	공공 API·업로드 파일·URL의 원본을 그대로 수집. 수집 시각·파라미터·건수·체크섬 기록. 파라미터 조합별 체크포인트	kpubdata, URL 수집기	외부 원본 → `bronze/{source}/raw_records.jsonl`, `metadata.json`
Silver (정제·검증)	명세에 선언한 규칙만 적용: 결측 토큰 → 컬럼 병합 → 이름 변경 → 자리 채움 → 형 변환 → 파생 컬럼. 필수 컬럼·자료형 검증, 통계·미리보기 생성, 개인정보·품질 검사	DuckDB	Bronze → `silver/{source}/table.parquet`, `schema.json`, `stats.json`, `validation.json`
Gold (패키징)	행 필터와 컬럼 선택, 선언한 개인정보 컬럼 마스킹, 분할, 두 소스 조인	DuckDB	Silver → `gold/{name}/table.parquet`, `package.json`, `splits/*.parquet`
Export (내보내기)	Gold를 exports 설정에 맞춰 파일로 저장하고 데이터셋 카드 생성	Exporter 6종 (csv, jsonl, parquet, markdown, huggingface, kaggle)	Gold → 산출물 파일, `README.md`, `card.json`
웨어하우스 커밋	상태가 ok인 실행의 Gold를 읽기 전용 스냅샷으로 커밋. 최근 3개 유지	SQLite 카탈로그, Parquet	Gold → `tables/{id}/snapshots/{id}/`

- 주요 선언 항목: 수집 파라미터(params·param_grid), 정규화 규칙(schema), 품질 기준(quality), 분할(splits), 내보내기(exports)
- Silver는 행과 컬럼을 모두 보존하고, 선택과 필터는 Gold에서만 수행

프로그램 목록 - KPubData Builder

기능ID	패키지·모듈	역할
A02_B01~02	`spec/`	BuildSpec 모델, 로더, 검증기, 스냅샷, 지문
A02·A03	`pipeline/`	오케스트레이터(`run_build`), 실행 문맥, 취소, 미리보기, 내보내기
A01_B01~02	`stages/bronze/`, `ingestion/`	소스 수집, 체크포인트, 파일·URL 읽기
A02_B05~06	`stages/silver/`, `quality/`, `tabular/`	DuckDB 정규화, 검증, 통계, 개인정보 검사, 스키마 변화 감지, 품질 평가
A02_B07	`stages/gold/`	선택, 마스킹, 분할, 조인, 카드
A03_B02	`exporters/`	csv, jsonl, parquet, markdown, huggingface, kaggle
A03_B05	`publishers/`	local, huggingface, kaggle
A03_B03	`manifest/`, `events/`	Manifest 모델·기록, 실행 이벤트와 종료 기록
A03_B04	`warehouse/`	테이블 카탈로그, 스냅샷 커밋·정리·백업
A06_B03	`query/`	읽기 전용 SQL 검사, 격리 실행, 집계, 프로파일
A04·A08	`service/`	HTTP 서비스, 경로, 인증, 비동기 작업, 게시, 모니터링, 관리
A06_B04	`credentials/`, `uploads/`	자격 증명 암호화 저장, 업로드 파일 저장
-	`store/`	빌드 인덱스, 상태 목록, 백업
A01_B04	`agent/`, `verify/`	신규 데이터셋 탐색·승인 확인, 명세 실검증
-	`cli.py`	명령줄 진입점 (validate, preview, build, publish, serve 등)

주요 CLI 명령

명령	용도
`validate` · `preview` · `build`	명세 검증, 저장 없는 미리보기, 파이프라인 실행
`publish`	산출물을 로컬 또는 원격 대상으로 게시
`serve`	Builder HTTP 서비스 실행
`rebuild-index` · `backup` · `restore`	인덱스 재구성, 상태 백업과 복원
`warehouse-gc` · `warehouse-backup`	스냅샷 정리와 백업
`prune-runs` · `prune-cancelled`	보존 기간이 지난 실행 정리

프로그램 목록 - KPubData Studio

기능ID	디렉터리	역할
A04_B01	`features/home`, `onboarding`	홈 데이터, 첫 실행 체크리스트
A04_B02	`features/discover`, `add-data`, `preview`, `validation`	카탈로그, 테이블 만들기 마법사, 미리보기·검증 클라이언트
A04_B02	`features/build-spec`	BuildSpec 변환, YAML, 초안, 비교, 개정 이력
A04_B03	`features/datasets`, `data-table`	테이블 목록·상세, 공통 표, 행 조회
A04_B04	`features/sql`, `charts`, `export`	SQL 실행, 저장된 분석, 집계 차트, 결과 내보내기
A04_B05	`features/reports`, `workspace`	리포트 저장·편집·내보내기, 최근 작업
A04_B06	`features/runs`, `artifacts`, `publish`	비동기 실행과 상태 조회, 산출물, 게시
A04_B07	`features/provider`, `version-check`	연결 관리, Builder 버전 비교
A06_B02	`features/licence`	이용 조건 요약
A06_B04	`features/auth`	Keycloak 연동, 토큰 갱신, 로그인 보호
A07_B01	`features/assistant`	Ask KPubData: 프롬프트, 구조 검증, 근거 대조, 동작, 비밀값 제거
A08	`features/monitoring`, `quality`, `admin`	모니터링, 품질 센터, 관리
-	`shared/lib/builderApi.ts`	Builder API 클라이언트. 모든 응답을 Zod 스키마로 검증
-	`pages/`, `app/`	화면 23종, 라우터, 공통 레이아웃

개발 스택

구분	내용
Builder	Python 3.10~3.13, DuckDB, Parquet(pyarrow), SQLite, sqlglot
Studio	React 19, TypeScript 6, Vite 8, React Router 7, Zustand 5, Tailwind CSS 4, Zod 4, i18next (한국어·영어)
인증	Keycloak (OIDC Authorization Code + PKCE), keycloak-js
테스트	Builder: pytest, ruff, mypy / Studio: Vitest, Playwright, axe-core
배포	Docker 이미지, GitHub Actions

테이블 정의서 - BuildSpec

○ BuildSpec 필드 구조 (dataset_id / title / description / sources[] 는 필수)

필수/선택	필드명	타입	설명
필수	`dataset_id`	string	데이터셋 식별자
필수	`title`	string	사람이 읽는 제목
필수	`description`	string	목적과 데이터 설명
필수	`sources[]`	array<SourceRef>	입력 소스 (1개 이상)
선택	`exports[]`	array<ExportTarget>	내보내기 대상 (kind·output_path·options). 비워두면 테이블 커밋까지만 수행
선택	`splits`	SplitSpec	분할 정의 (mode: ratio | key, ratios, key, seed)
선택	`quality`	QualityPolicy	품질 기준 (중복률, 결측률, 최소 행 수, 값 범위, 컬럼 비교)
선택	`pii`	PiiPolicy	개인정보 검사 정책 (mode: block | warn | allow)
선택	`composition`	CompositionSpec	두 소스를 조인해 하나의 Gold 생성
선택	`license` · `license_name` · `license_link` · `attribution`	string	라이선스와 출처 표기. 게시할 때 license 필수
선택	`publish`	boolean	실행 후 게시 여부 (기본 false)
선택	`refresh_cadence`	string	갱신 주기 (ISO-8601 기간, 예: P1D). 갱신 지연 판정에 사용
선택	`metadata`	object	산출물에 실을 임의 메타데이터

- ※ DB 테이블이 아닌 실행 계약의 필드 구조를 테이블 정의서 형식으로 정리함 (출처: `spec/models.py`)

테이블 정의서 - SourceRef

○ SourceRef 필드 구조 — BuildSpec의 sources[] 항목

필드명	타입	설명
`kind`	string	소스 종류: public_api(기본) | file | url
`provider · dataset`	string	kpubdata 제공 기관과 데이터셋 (public_api에서 필수)
`params · param_grid`	object	수집 파라미터, 조합별로 수집할 파라미터 축
`alias`	string	출력 키 이름. 조인에서 참조
`schema`	SchemaContract	정규화·검증 계약 (required, dtypes, casts, rename, derived, null_tokens, coalesce, zfill)
`gold`	GoldSelection	Gold에 남길 컬럼과 행 (select, filters, pii_columns)
`upload_id · format · encoding`	string	파일 소스: 업로드 식별자, 형식(csv·json·jsonl·parquet), 인코딩
`endpoint · method`	string	URL 소스: 주소(HTTPS), 메서드(GET만 허용)
`allow_empty`	boolean	0행 결과가 기존 테이블을 대체하도록 허용할지 여부

○ SchemaContract 적용 순서

- read_as → null_tokens → coalesce → rename → zfill → casts → derived
- 형 변환에서 값이 사라지면 오류로 처리하며, 선언하지 않은 변환은 수행하지 않음
- ※ 출처: `spec/models.py`

테이블 정의서 - BuildManifest

○ BuildManifest 필드 구조 — 파이프라인에 들어간 실행마다 1건 생성 (schema_version 1.0.0)

필드명	타입	설명
build_id	string	실행 식별자
started_at / finished_at	datetime	실행 시작·종료 시각
status · partial	string · bool	ok | failed | cancelled. partial은 취소된 실행에서만 true
inputs / outputs	tuple<string>	소스 출력 키 목록 / 생성된 산출물 경로 목록
warnings / errors	tuple<string>	경고 / 실패 메시지
failures	tuple<RunFailure>	실패 내역 (소스, 단계, 코드, 요약)
row_counts	dict<str,int>	소스·조인별 Silver 행 수
schema_summaries	dict<str,SchemaSummary>	소스별 스키마 요약
provenance	tuple<SourceProvenance>	소스별 수집 시각, 파라미터, 레코드 수, 체크섬
quality_results	dict	평가한 품질 검사 결과 (pass 포함)
schema_drift	dict	이전 실행 대비 스키마·행 수 변화
gold_selection · pii_masking	dict	Gold 선택 규칙과 행 수, 마스킹한 컬럼
composition	CompositionProvenance	조인 내역
warehouse_failures	dict	커밋에 실패한 테이블과 사유
build_environment	BuildEnvironment	Python·kpubdata·Builder 버전
inputs_fingerprint	string | None	입력 데이터 지문 (sha256:…)
artifacts	dict	Gold 디렉터리별 산출물 다이제스트
created_by · retry_of	string | None	요청자 표시 이름, 재시도 대상 실행

- ※ 출처: manifest/models.py

테이블 정의서 - 저장 인덱스

○ builds 표 (SQLite 인덱스, `_builds.sqlite`)

컬럼	타입	설명
`run_id`	TEXT PK	실행 식별자
`status`	TEXT NOT NULL	ok | failed | cancelled (CHECK 제약)
`started_at · finished_at`	TEXT	시작·종료 시각. finished_at 내림차순 인덱스
`spec_digest`	TEXT	명세 다이제스트
`dataset_id`	TEXT	데이터셋 식별자 (인덱스)
`created_by · owner_id`	TEXT	요청자, 소유자
`error · error_code · error_stage`	TEXT	실패 요약, 코드, 단계

○ SQLite 저장소 목록

파일	표	용도
`_builds.sqlite`	builds	실행 인덱스 (Manifest에서 재구성 가능)
`_build_events.sqlite`	build_events, run_submissions, run_endings	실행 이벤트, 제출 기록, Manifest 없이 끝난 실행의 종료 기록
`_warehouse.sqlite`	tables, table_snapshots, snapshot_leases, snapshot_holds	웨어하우스 카탈로그와 스냅샷 보존
`_publish_receipts.sqlite`	publish_receipts, publish_receipt_audit	게시 영수증과 감사 기록
`.service/analyses.sqlite3`	analyses	저장된 분석
`.service/revisions.sqlite3`	revisions, revision_audit	명세·주석 개정 이력
`.service/uploads.sqlite3`	uploads	업로드 파일 메타데이터
`.service/provider-credentials.sqlite3`	provider_credentials	제공 기관 자격 증명 (암호화)
`.service/users.sqlite3`	users	가입 신청 원장

- ※ 출처: `store/build_index.py`, `store/inventory.py`

핵심 소스코드(1) — kpubdata-builder · run_build()

```
def run_build(spec: BuildSpec, *, client: SourceClient, output_root: Path,
              run_id: str | None = None, retry_of: str | None = None,
              catalog: TableCatalog | None = None, ...) -> BuildResult:
    validate_spec(spec)                                  # 위반 시 ValidationError, 기록 없음
    context = BuildContext.create(spec, output_root=output_root, run_id=run_id)
    ...
    # Per-source fetch/stage mostly waits on network I/O, so concurrent
    # execution via thread pool reduces total time (#247). executor.map
    # returns results in spec.sources order, not completion order, so
    # downstream merge (manifest) stays deterministic.
    max_workers = min(len(spec.sources), _MAX_PARALLEL_SOURCES)
    clear_temp_root(context.output_root / context.run_id)
    try:
        with ThreadPoolExecutor(max_workers=max_workers) as executor:
            results = list(executor.map(_worker, spec.sources))
    ...
    if cancelled:
        status = "cancelled"
    elif errors:
        status = "failed"
        recorder.run_failed(failed_source_count=len(errors))
    else:
        status = "ok"
        recorder.run_finished()
    ...                                                  # ok일 때 웨어하우스 커밋, manifest.json 기
록
```

- BuildSpec을 검증한 뒤 실행 작업공간을 만들고, 소스별 Bronze→Silver→Gold 파이프라인을 병렬로 실행해 상태를 판정하는 핵심 로직
- 결과는 명세 순서대로 모아 Manifest가 실행마다 같은 순서로 기록되게 함
- `...`은 생략한 부분. 원본: `src/kpubdata_builder/pipeline/orchestrator.py`

핵심 소스코드(2) — kpubdata-builder · validate_spec()

```
def validate_spec(spec: BuildSpec) -> None:
    """Validate BuildSpec minimum executability.

    Raises:
        ValidationError: When one or more validation rules fail.
    """
    problems: list[ValidationProblem] = []
    if not spec.dataset_id.strip():
        problems.append(_p("empty_field", "dataset_id", "dataset_id must be a non-empty
string"))
    if not spec.title.strip():
        problems.append(_p("empty_field", "title", "title must be a non-empty string"))
    if not spec.description.strip():
        problems.append(_p("empty_field", "description", "description must be a non-empty
string"))
    if not spec.sources:
        problems.append(_p("missing_sources", "sources", "at least one source is required"))
    ...
    # exports may be empty (#703). A build with none ends at a committed table,
    # which is a complete job — publishing is an explicit follow-up, not the only
    # way to finish.
    ...
    problems.extend(_split_problems(spec))
    problems.extend(_schema_problems(spec))
    problems.extend(_source_kind_problems(spec))
    problems.extend(_source_key_problems(spec))
    problems.extend(_param_grid_problems(spec))
    problems.extend(_pii_problems(spec))
    problems.extend(_license_problems(spec))
    problems.extend(_quality_problems(spec))
    problems.extend(_composition_problems(spec))
    problems.extend(_gold_selection_problems(spec))
    if problems:
        raise ValidationError([str(p) for p in problems], structured=problems)
```

- 실행 자원을 만들기 전에 명세의 문제를 모두 모아 한 번에 알려 주는 사전 검증 로직
- 반환값은 없으며, 문제가 있으면 code·path·message·hint를 담은 `ValidationError` 발생
- `exports`는 비어 있어도 되며, 이 경우 테이블 커밋까지만 수행
- `...`은 생략한 부분. 원본: `src/kpubdata_builder/spec/validator.py`

핵심 소스코드(3) — kpubdata-studio · 근거 대조

```
export function crossCheckAssistantResponse(
  response: AssistantStructuredResponse,
  evidence: AssistantEvidence,
  knownRefs: AssistantKnownRefs,
): CrossCheckResult {
  const rejectedRefs: string[] = [];
  const evidenceRefs = response.evidenceRefs.filter((ref) => {
    const known = isKnownEvidenceRef(ref, knownRefs, evidence);
    if (!known) rejectedRefs.push(`${ref.kind}:${ref.id} (${ref.label})`);
    return known;
  });

  const rejectedActions: string[] = [];
  const suggestedActions = response.suggestedActions.filter((action) => {
    const check = isKnownAction(action, knownRefs, evidence);
    if (!check.ok) rejectedActions.push(check.reason);
    return check.ok;
  });

  let generatedSql = response.generatedSql;
  let rejectedSqlReason: string | undefined;
  if (generatedSql) {
    if (evidence.context.stage !== generatedSql.stage) {
      rejectedSqlReason = t("sqlStageMismatch", { ... });
      generatedSql = null;
    } else if (generatedSql.source && !knownRefs.sourceKeys.has(generatedSql.source)) {
      ...                                                    // 소스가 하나면 소스 지정만 제거, 여럿이면 SQL
폐기
    }
  }
  ...
}
```

- LLM 답변이 인용한 근거·제안 동작·SQL을 Builder에서 받은 실제 근거와 대조하는 로직
- 근거에 없는 항목은 그 항목만 버리고 나머지 답변은 유지
- SQL은 현재 단계·소스와 맞지 않으면 버리며, 실행 전 최종 검사는 Builder가 수행
- `...`은 생략한 부분. 원본: `src/features/assistant/crossCheck.ts`

참고

- KPubData Builder 기준 코드: 0539bca (2026-10-12, API 계약 1.123.0)
- KPubData Studio 기준 코드: 08e3e8a (2026-10-12, 버전 0.4.0)
- 화면 그림은 Studio를 데모 모드로 실행해 캡처한 것으로, 표시된 값은 데모 데이터임
- 기술 설명별 코드 근거: 근거 목록
- 2026. 09. 08 판 대비 변경 사항: 제작설계서 변경 내역